VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

ASSIGNMENT II

DEEP LEARNING AND ITS APPLICATIONS — CO3133

DEEP LEARNING ON LARGE-SCALE DATA AND SPECIALIZED TASKS

Deep Learning on Large-Scale Data for Specialized Computer Vision or Language Tasks.

MAJOR: COMPUTER SCIENCE

CLASS: CC04

SUPERVISOR: Lê Thành Sách

— o0o —

STUDENT: HOÀNG XUÂN BÁCH 2352082
NGUYỄN VIỆT HÙNG 2352424
LÊ NGUYỄN GIA PHÚC 2352931

HO CHI MINH CITY, October 2026

Contents

1 Problem and Data Description **1**

1.1 Problem formulation . . . 1

1.1.1 Real-world motivation . . . 1

1.1.2 Input . . . 1

1.1.3 Output . . . 1

1.1.4 Prediction unit and mathematical formulation . . . 1

1.1.5 Main difficulties . . . 2

1.2 Dataset description . . . 2

1.2.1 Source, version, and license . . . 2

1.2.2 Collection and labeling pipeline . . . 2

1.2.3 Sample counts and annotation format . . . 3

1.2.4 Storage size and memory footprint . . . 3

1.2.5 Dataset bias and limitations . . . 3

1.3 Exploratory Data Analysis . . . 4

1.3.1 Input size . . . 4

1.3.2 Class distribution . . . 5

1.3.3 Imbalance analysis . . . 6

1.3.4 Pixel-intensity distribution (training split only) . . . 7

1.3.5 Representative samples . . . 7

1.3.6 Summary . . . 9

1.4 Data splitting . . . 9

1.4.1 Seed and Deterministic Reproducibility . . . 9

1.4.2 Split Unit and Stratification . . . 9

1.4.3 Data Leakage Prevention . . . 9

1.4.4 Subset Selection Rules . . . 11

1.5 Data Preprocessing . . . 11

1.5.1 Data Cleaning and Invalid Annotation Handling . . . 11

1.5.2 Image Resizing, Padding, and Normalization . . . 11

1.5.3 Data Augmentation . . . 12

1.5.4 Input Tensor Shapes . . . 12

1.5.5 Example Batch after Preprocessing . . . 12

2 Methodology **14**

2.1 Pipeline Overview . . . 14
2.2 Convolutional Neural Network (CNN) . . . 14
2.2.1 Architecture and input representation . . . 14
2.2.2 Parameter Budget Analysis . . . 15
2.2.3 Strengths and limitations . . . 16
2.2.3.a Strengths . . . 16
2.2.3.b Limitations . . . 16
2.2.4 Self-implemented components and library dependencies . . . 17
2.3 Residual Network (Resnet-18) . . . 17
2.3.1 Residual Learning and Input Representation . . . 17
2.3.2 ResNet-18 . . . 18
2.3.3 Parameter Budget Analysis. . . . 19
2.3.4 Strengths and limitations . . . 19
2.3.4.a Strengths . . . 19
2.3.4.b Limitations . . . 20
2.3.5 Self-implemented components and library dependencies . . . 20
2.4 Shared Loss and evaluation metrics . . . 20
2.4.1 Loss function . . . 20
2.4.2 Predictive evaluation metrics . . . 21
2.5 Training setup . . . 23
2.6 Experimental Design . . . 25
2.6.1 Experiment 1: Model Configuration Comparison . . . 25
2.6.2 Experiment 2: Horizontal-flip Augmentation Ablation . . . 26

3 Implementation Results **28**
3.1 Training Behavior . . . 28
3.1.1 Baseline (SimpleCNN) . . . 29
3.1.2 frozen-backbone ResNet-18 . . . 30
3.1.3 ResNet-18 Full Fine-Tuning . . . 31
3.2 Quantitative Results . . . 32
3.2.1 Overall Performance and Computational Cost . . . 32
3.2.2 Per-Class Performance . . . 33
3.2.3 Test Confusion Matrices . . . 35
3.3 Qualitative Results . . . 37
3.3.1 Correct Predictions . . . 37

3.3.2 Difficult but Correct Predictions . . . 38
3.3.3 Failure Cases . . . 40
3.4 Comparison and Discussion . . . 41
3.4.1 Overall Predictive Performance . . . 41
3.4.2 Class-Specific Improvements and Remaining Errors . . . 41
3.4.3 Architecture and Input Representation . . . 41
3.4.4 Loss, Training Behavior, and Prediction Confidence . . . 42
3.4.5 Accuracy, Execution Time, and Parameter Trade-offs . . . 42
3.4.6 Hypothesis Assessment and Limitations . . . 42
3.5 Error Analysis . . . 42
3.5.1 Confusion Between Visually Similar Classes . . . 42
3.5.2 Incorrect Predictions with High Softmax Scores . . . 45
3.6 Ablation Study . . . 46
3.7 Limitations and Conclusion . . . 46
3.7.1 Data Limitations . . . 46
3.7.2 Method limitations. . . . 47
3.7.3 Generalization and Future Work . . . 47
3.7.4 Conclusion . . . 47

4 AI Usage Disclosure **49**
4.1 Review and Refinement of the Training and Evaluation Pipeline . . . 49
4.2 Model Architecture Review and Technical Verification . . . 50
4.3 Feedback on Results Interpretation and Discussion . . . 50
4.4 English Language Editing . . . 51
4.5 AI-Assisted Github Pages Skeleton . . . 52
4.6 Consistency Verification Between Experimental Outputs and Report . . . 52
4.7 Consistency Verification Between Experimental Outputs and GitHub Pages . . . 53
4.8 LaTeX Formatting Assistance . . . 54

List of Figures

1 Training-set RGB pixel intensities with padding excluded. The linear plot marks the channel means, and the logarithmic plot shows the peaks at intensities 0 and 255. 7

2 Seven randomly selected training images per Food-11 class, shown without augmentation. 8

3 Examples of exact duplicates detected by MD5 within and across partitions. Dairy product examples are shown on the left and Dessert examples on the right. 10

4 Examples of visually similar image pairs detected by pHash within and across partitions. 10

5 Pipeline Overview 14

6 Training and validation loss, accuracy, and Macro-F1 of SimpleCNN across seeds 42, 43, and 44. Dotted lines and markers identify the validation-selected checkpoints. 29

7 Training and validation learning curves of frozen-backbone ResNet-18 across three seeds. Each seed's selected epoch is shown in the legend and marked on the curves. 30

8 Training and validation learning curves of fully fine-tuned ResNet-18 across three seeds. Selected checkpoints are marked separately for each seed. 31

9 Food-11 test confusion matrices for SimpleCNN. 35

10 Food-11 test confusion matrices for ResNet-18 with a frozen backbone. 36

11 Food-11 test confusion matrices for ResNet-18 with full fine-tuning. 36

12 Correct predictions by SimpleCNN for training seed 42. 37

13 Correct predictions by ResNet-18 with a frozen backbone for training seed 42. 37

14 Correct predictions by fully fine-tuned ResNet-18 for training seed 42. 38

15 Correct predictions with small top-two softmax score gaps for SimpleCNN, training seed 42. 38

16 Correct predictions with small top-two softmax score gaps for ResNet-18 with a frozen backbone, training seed 42. 39

17 Correct predictions with small top-two softmax score gaps for fully fine-tuned ResNet-18, training seed 42. 39

18 Misclassified test images from SimpleCNN for training seed 42. 40

19 Misclassified test images from ResNet-18 with a frozen backbone for training seed 42. . 40

20 Misclassified test images from fully fine-tuned ResNet-18 for training seed 42. 41

21 G1: Dairy product images predicted as Dessert, including mixed cheese plates and spread-covered bread. 44

22 G2: Mixed dishes containing bread alongside eggs or meat, illustrating competing class cues within a single image. 44

23 G3: Fried food, Seafood, and Meat images predicted as Dessert. The churro examples illustrate overlap between preparation-based and dessert categories. 44
24 G4: Dessert-labeled images predicted as Meat, Egg, or Soup. The second example contains a visible egg and warrants review of its annotation. 45
25 G5: Confusion involving Fried food, Meat, and Egg, with overlapping preparation and ingredient cues. 45

List of Tables

1 Food-11 dataset overview and distribution sources. 2
2 Food-11 split sizes and their roles in the experiments. 3
3 Ten most common resolutions in the original training split. 4
4 Original training images by number of dimensions above 512 pixels. Shares are rounded independently. 4
5 Class distribution before duplicate removal. Each percentage is relative to the original size of its split. 6
6 Original class imbalance by partition. The largest-class count and share apply to each of Dessert and Soup separately. 6
7 Training-set pixel statistics and ImageNet normalization constants on the $[0, 1]$ scale. . . 7
8 Class distribution after duplicate removal. Each percentage is calculated relative to the number of retained images in its split. 11
9 Layer-by-layer trainable parameter breakdown of `SimpleCNN`. 15
10 ResNet-18 backbone dimensions for the configured input resolution. 18
11 Parameter breakdown of the adapted ResNet-18. Stage counts include projection shortcuts where present. 19
12 Training and Optimization Hyperparameters for SimpleCNN and ResNet-18 24
13 Training and Optimization Hyperparameters for SimpleCNN and ResNet-18 25
14 Predictive performance, model size, and computational cost of SimpleCNN and the two ResNet-18 configurations on Food-11. Performance and timing results are reported as mean $\pm$ sample standard deviation across training seeds 42, 43, and 44. Macro-F1 is the primary predictive metric. Bold values indicate the best mean predictive results. 32
15 Per-class test metrics of SimpleCNN on Food-11, reported as mean $\pm$ sample standard deviation across training seeds 42, 43, and 44. Bold and underlined values indicate the highest and lowest means, respectively, within each metric column. 33

16 Per-class test metrics of ResNet-18 with a frozen backbone on Food-11, reported as mean ± sample standard deviation across training seeds 42, 43, and 44. Bold and underlined values indicate the highest and lowest means, respectively, within each metric column. . 34
17 Per-class test metrics of ResNet-18 with full fine-tuning on Food-11, reported as mean ± sample standard deviation across training seeds 42, 43, and 44. Bold and underlined values indicate the highest and lowest means, respectively, within each metric column. . 34
18 Error groups among the 242 incorrect test predictions of fully fine-tuned ResNet-18, seed 42. Percentages are relative to all errors. 43
19 Test performance without horizontal flipping, reported as mean ± sample standard deviation across training seeds 42, 43, and 44. Bold values indicate the best mean for each metric. 46
20 Paired test differences computed as Augmented minus No Augmentation within each training seed. Results are reported as mean ± sample standard deviation across seeds 42, 43, and 44. Differences in accuracy and macro-averaged metrics are in percentage points (%). 46

1. Problem and Data Description

1.1. Problem formulation

1.1.1. Real-world motivation

Automatic food recognition can reduce the effort required to organize meal photographs and maintain food diaries. A classifier can provide an initial food category for a user to review, supporting the first stage of a dietary logging system. This assignment studies that recognition step using Food-11. The task is limited to identifying a broad food category from an image, the available labels do not support estimating ingredients, portion sizes, calories, or nutritional intake.

The experimental question is whether ImageNet transfer learning improves Food-11 classification over a SimpleCNN trained from scratch, and how backbone trainability and capacity affect predictive performance and computational cost. The notebook defines frozen-backbone and full-fine-tuning variants of ResNet-18 and ResNet-50 under a shared data protocol.

1.1.2. Input

Each raw input is one food photograph with variable width and height. The observed files are RGB JPEG images. Before entering a model, the image is converted to RGB, resized with its aspect ratio preserved, centrally padded, and normalized. The resulting input is a floating-point tensor $x \in \mathbb{R}^{3\times256\times256}$ in channel-first order followed by height and width. Training also applies the flipping transformations described below.

1.1.3. Output

The classifier produces 11 logits, one for each class in Table 8. A softmax converts these logits to a probability vector, and the class with the largest probability is returned as the prediction. Class indices are assigned by sorting the observed class names alphabetically, yielding indices $0,\dots,10$ in the table's row order.

1.1.4. Prediction unit and mathematical formulation

The prediction unit is the *entire image*, with exactly one category label. This is supervised, single-label, multiclass image classification. For a training set $\mathcal{D}_{\text{tr}} = \{(x_i, y_i)\}_{i=1}^{9866}$, where $y_i \in \{0,\dots,10\}$, the model computes logits $z_\theta(x_i)$ and

$$p_\theta(y = k \mid x_i) = \frac{\exp(z_{\theta,k}(x_i))}{\sum_{j=0}^{10}\exp(z_{\theta,j}(x_i))}, \qquad \widehat{y}_i = \underset{k\in\{0,\dots,10\}}{\arg\max}\ p_\theta(y = k \mid x_i). \tag{1}$$

Training uses ordinary, unweighted cross-entropy without label smoothing:

$$\mathcal{L}(\theta) = -\frac{1}{N_{\text{tr}}} \sum_{i=1}^{N_{\text{tr}}} \log p_\theta(y = y_i \mid x_i). \tag{2}$$

1.1.5. Main difficulties

Food appearance varies with preparation, viewpoint, illumination, scale, and background. A broad class can contain visually different dishes, while different classes can share similar colors and textures. Category boundaries can also overlap: a fried seafood dish, for example, has attributes associated with both Fried food and Seafood even though the dataset supplies one label. Other challenges include unequal class frequencies, variable source image sizes, and cross-split duplicate images. The duplicate audit limits how strongly benchmark results can be interpreted as generalization to unseen photographs.

1.2. Dataset description

1.2.1. Source, version, and license

Food-11 was released by the Multimedia Signal Processing Group (MMSPG) at EPFL for the food categorization task studied by Singla, Yuan, and Ebrahimi (MADiMa 2016). The experiments use version 1 of the Kaggle mirror, downloaded with `kagglehub`. Table 1 distinguishes the original release from the copy used for reproducibility.

Table 1: Food-11 dataset overview and distribution sources.

Field	Description
Dataset	Food-11, comprising 16,643 RGB food images in 11 categories
Original source	MMSPG at EPFL, which provides the dataset for research use
Kaggle mirror	`trolukovich/food11-image-dataset`
Mirror publisher and version	Aleksandr Antonov, version 1
Mirror license listing	CC0 (Public Domain), as listed on Kaggle

1.2.2. Collection and labeling pipeline

Food-11 groups food photographs into 11 broad categories. The Kaggle mirror preserves the predefined `training/`, `validation/`, and `evaluation/` partitions and organizes each partition into one subfolder per class. The experiments use the supplied class assignments. The loader reads the class subfolder name as the label and assigns indices 0–10 in alphabetical order. The same class-to-index mapping is checked across all three partitions.

Each category covers several kinds of dish. For example, the Bread samples include toast, pizza, and burgers (Figure 2). Images can contain multiple foods, so a single class label simplifies the photographed scene.

1.2.3. Sample counts and annotation format

The original release contains 16,643 RGB JPEG images with variable width and height. Each image has one image-level categorical label encoded as an integer in $\{0, \ldots, 10\}$. The supplied annotations support single-label classification of the whole photograph.

Table 2 gives the predefined partition sizes and the counts used for the reported experiments after duplicate removal (Section 1.4). Split percentages are based on the 16,643 original images. Table 5 reports the original counts by class.

Table 2: Food-11 split sizes and their roles in the experiments.

Split	Original	Share	Role
Training	9,866	59.28%	Model fitting
Validation	3,430	20.61%	Model and checkpoint selection
Evaluation	3,347	20.11%	Final test evaluation
Total	16,643	100.00%	

1.2.4. Storage size and memory footprint

The original image files occupy approximately 1,133 MiB on disk, comprising 643 MiB for training, 253 MiB for validation, and 238 MiB for evaluation. These split sizes are rounded separately.

After resizing and padding, each RGB image contains $3 \times 256 \times 256 = 196{,}608$ values. This requires 192 KiB in an 8-bit representation or 768 KiB as an FP32 tensor. Storing all 9,866 original training inputs in FP32 would take about 7.23 GiB, and storing all 16,643 inputs would take about 12.19 GiB. The data loader decodes and transforms images batch by batch. A batch of 128 images occupies 96 MiB for the FP32 input tensor, before model activations, gradients, and optimizer state.

1.2.5. Dataset bias and limitations

- **Class imbalance.** The largest original training classes have about 5.36 times as many images as Rice. This motivates macro-F1 as the primary metric (Section 1.3.3).
- **Broad and overlapping categories.** A category can contain visually different dishes, and different categories can share colors, textures, and preparation styles. Mixed plates can contain several food types while carrying a single label.
- **Annotation ambiguity.** Some representative images contain background subjects or off-topic content, including a shop sign assigned to Dessert. These examples illustrate potential label noise, whose overall frequency is unquantified.
- **Photo-style bias.** Studio backgrounds in some Dairy product and Vegetable/Fruit examples differ from the restaurant photographs in other classes. Models may use these visual cues together with

the food appearance.

- **Duplicates and grouping metadata.** Duplicate images require cleaning within and across the predefined partitions (Section 1.4). The available labels identify food classes. The dataset supplies no subject, restaurant, plate, or scene identifier for grouped splitting.

1.3. Exploratory Data Analysis

The class distribution describes all three original partitions. The resolution analysis used to choose the preprocessing rule, the pixel-intensity statistics, and the representative samples come from the original training partition before duplicate removal.

1.3.1. Input size

Table 3 lists the ten most common native resolutions among the 9,866 training images, expressed as width $\times$ height. The most common resolution is 512×512 (5,551 images, 56.26%). The next two most common resolutions, 512×384 and 384×512, have the same longer side and different orientations.

Table 3: Ten most common resolutions in the original training split.

Rank	Resolution (pixels)	Images	Share
1	512×512	5,551	56.26%
2	512×384	1,355	13.73%
3	384×512	519	5.26%
4	382×512	320	3.24%
5	512×382	216	2.19%
6	512×341	142	1.44%
7	1024×768	108	1.09%
8	512×306	107	1.08%
9	512×289	81	0.82%
10	511×512	60	0.61%
	Other	1,407	14.26%
	Total	9,866	100.00%

Table 4 groups the same images by how many dimensions are strictly greater than 512 pixels. In total, 547 images (5.54%) exceed 512 pixels in at least one dimension.

Table 4: Original training images by number of dimensions above 512 pixels. Shares are rounded independently.

Dimensions above 512 pixels	Images	Share
Neither dimension	9,319	94.46%
Exactly one dimension	120	1.22%
Both dimensions	427	4.33%
Total	9,866	100.00%

We chose a common input size of 256×256 to balance spatial detail and compute cost across models and repeated runs. For the dominant 512×512 group, this halves each dimension and reduces the input

pixel count from 262,144 to 65,536 (75% fewer pixels). The preprocessing rule was fixed before training using the original training images and applied to all partitions.

Each image is resized by bilinear interpolation. For original width w and height h, the scale is $s = 256/\max(w,h)$ and the resized dimensions are

$$w' = \max(1, \text{round}(sw)), \qquad h' = \max(1, \text{round}(sh)).$$

The image is centered on a black 256×256 RGB canvas. Padding is divided as evenly as possible between opposite sides. For example, 512×384 becomes 256×192 with 32 pixels of padding above and below. A 384×512 image receives 32 pixels on the left and right after resizing to 192×256. Square images fill the canvas.

The shortest longer side in the original training split is 270 pixels, so all training images are downsampled. After rounding the resized dimensions, 4,211 images (42.68%) require padding. Padding covers 11.72% of the canvas on average across the original training images, including images with no padding.

1.3.2. Class distribution

Table 5 gives the number of images per class in the original predefined partitions. Each percentage is calculated relative to the size of its own split. Dessert and Soup each contribute 1,500 training images (15.20% each), followed by Meat with 1,325 (13.43%). Rice, Dairy product, and Noodles/Pasta have the fewest training images. Validation and evaluation have similar class proportions. The counts describe the original dataset before duplicate removal, with the resulting partition sizes given in Table 2.

Table 5: Class distribution before duplicate removal. Each percentage is relative to the original size of its split.

		Training		Validation		Evaluation		
Idx	Class	Images	%	Images	%	Images	%	Total
0	Bread	994	10.08	362	10.55	368	10.99	1,724
1	Dairy product	429	4.35	144	4.20	148	4.42	721
2	Dessert	1,500	15.20	500	14.58	500	14.94	2,500
3	Egg	986	9.99	327	9.53	335	10.01	1,648
4	Fried food	848	8.60	326	9.50	287	8.57	1,461
5	Meat	1,325	13.43	449	13.09	432	12.91	2,206
6	Noodles/Pasta	440	4.46	147	4.29	147	4.39	734
7	Rice	280	2.84	96	2.80	96	2.87	472
8	Seafood	855	8.67	347	10.12	303	9.05	1,505
9	Soup	1,500	15.20	500	14.58	500	14.94	2,500
10	Vegetable/Fruit	709	7.19	232	6.76	231	6.90	1,172
	Total	9,866	100	3,430	100	3,347	100	16,643

1.3.3. Imbalance analysis

The original training split has a largest-to-smallest class ratio of $1{,}500/280 \approx 5.36$, with Dessert and Soup tied for the largest class and Rice the smallest. Table 6 shows the same pattern in the other partitions. Across the whole dataset, each largest class contains 2,500 images (15.02%), compared with 472 (2.84%) for Rice.

Table 6: Original class imbalance by partition. The largest-class count and share apply to each of Dessert and Soup separately.

Split	Largest count	Share (%)	Rice count	Share (%)	Ratio
Training	1,500	15.20	280	2.84	5.36
Validation	500	14.58	96	2.80	5.21
Evaluation	500	14.94	96	2.87	5.21
All	2,500	15.02	472	2.84	5.30

Training uses unweighted cross-entropy, so more frequent classes contribute more samples to the loss. Macro-F1 is the primary evaluation metric because it gives each of the 11 classes equal weight. Top-1 accuracy, macro-precision, macro-recall, and per-class metrics provide additional context. For reference, always predicting Dessert or Soup would yield 14.94% accuracy on the original evaluation partition.

1.3.4. Pixel-intensity distribution (training split only)

Figure 1 shows RGB intensities from the original training images resized to a longer side of 256 pixels, before padding or augmentation. The means on the $[0, 255]$ scale are 142.4 (R), 115.3 (G), and 87.6 (B). Table 7 compares the training statistics with the fixed ImageNet normalization constants.

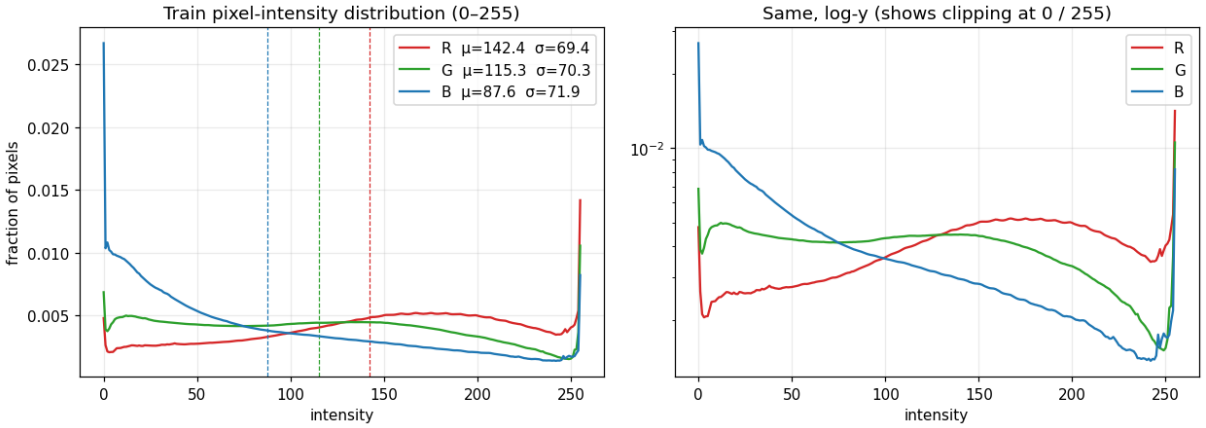

Figure 1: Training-set RGB pixel intensities with padding excluded. The linear plot marks the channel means, and the logarithmic plot shows the peaks at intensities 0 and 255.

Table 7: Training-set pixel statistics and ImageNet normalization constants on the $[0, 1]$ scale.

Channel	Food-11 mean	ImageNet mean	Food-11 std.	ImageNet std.
R	0.5586	0.485	0.2722	0.229
G	0.4520	0.456	0.2755	0.224
B	0.3434	0.406	0.2821	0.225

Table 7 shows that the Food-11 training images have a higher red-channel mean, a lower blue-channel mean, and larger standard deviations in all three channels than the ImageNet reference values. We use ImageNet normalization to match the channel-wise scaling used for the pretrained ResNet-18 weights. The same constants are applied to SimpleCNN and both ResNet-18 configurations across all splits, keeping normalization fixed in the model comparison and the horizontal-flip ablation. The Food-11 statistics are reported to describe the training data. This choice prioritizes compatibility with the pretrained models and consistency across runs.

1.3.5. Representative samples

Figure 2 shows seven unaugmented training images per class. Vegetable/Fruit spans salads, fruit, and market displays, while Bread, Fried food, and Egg can share golden-brown colors and textures. Mixed plates and a shop sign labeled Dessert illustrate ambiguity in the image-level labels. Studio backgrounds in some Dairy product images may also give models cues unrelated to the food.

Figure 2: Seven randomly selected training images per Food-11 class, shown without augmentation.

1.3.6. Summary

- **Input size.** Resolution analysis uses the original 9,866 training images. The most common resolution is 512×512 (56.26%), and 94.46% have neither dimension above 512 pixels. The shared 256×256 input balances spatial detail and compute cost. Aspect-preserving resizing followed by black padding retains the full image extent.
- **Class distribution and imbalance.** The original partitions contain 9,866 training, 3,430 validation, and 3,347 evaluation images. The largest training classes have about 5.36 times as many samples as Rice, supporting the use of macro-F1 as the primary metric.
- **Representative content.** Broad categories, mixed plates, photo-style differences, and ambiguous labels motivate per-class evaluation and confusion-matrix analysis.

1.4. Data splitting

1.4.1. Seed and Deterministic Reproducibility

The experiments use the predefined Food-11 training, validation, and evaluation partitions. No random repartitioning is performed, and therefore no split seed is required. Before model initialization and training, the training seed is assigned to Python's random-number generator, NumPy, and the PyTorch CPU and CUDA generators. A generator initialized with the same seed controls mini-batch shuffling. These seeds control stochastic operations such as parameter initialization, training augmentation, and dropout.

1.4.2. Split Unit and Stratification

The split unit is an individual image file, with partition membership inherited from the predefined Food-11 training, validation, and evaluation folders. No additional stratified splitting is performed

1.4.3. Data Leakage Prevention

Duplicate checks are performed within and across the predefined partitions before training. Removal is applied within each partition first, followed by cross-partition removal, with retention priority given to training, then validation, and finally evaluation. Label consistency was also checked to distinguish repeated images from identical images assigned to different categories. All images removed during deduplication shared the same class label as their retained counterparts.

- **MD5-based exact-duplicate detection:** MD5 checksums are computed from the image file contents. Files with identical checksums are treated as exact duplicates, and only one representative is retained. This check is performed before perceptual-hash-based removal.
- **perceptual-hash-based near-duplicate detection:** Images are compared using 64-bit perceptual

hashes and Hamming distance. Pairs with a distance of at most 10 are recorded as similarity candidates for inspection. During automatic removal, a stricter threshold of at most 5 is used to identify near-duplicate pairs. Two additional duplicate images identified through visual inspection are removed manually.

Figure 3: Examples of exact duplicates detected by MD5 within and across partitions. Dairy product examples are shown on the left and Dessert examples on the right.

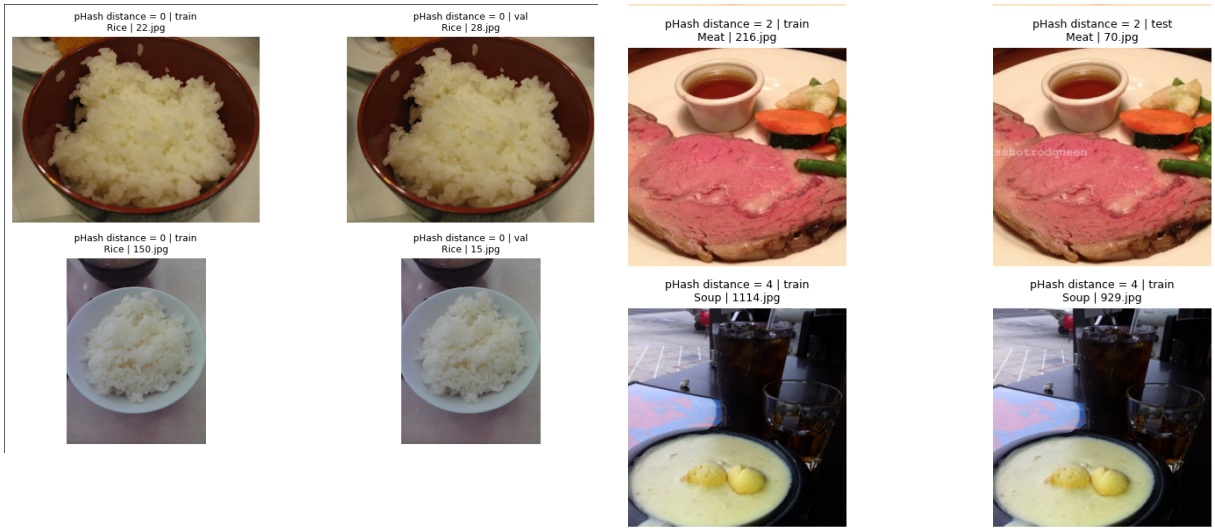

Figure 4: Examples of visually similar image pairs detected by pHash within and across partitions.

Table 8: Class distribution after duplicate removal. Each percentage is calculated relative to the number of retained images in its split.

		Training		Validation		Evaluation		
Idx	Class	Images	%	Images	%	Images	%	Total
0	Bread	993	10.09	362	10.76	268	9.68	1,623
1	Dairy product	423	4.30	141	4.19	147	5.31	711
2	Dessert	1,499	15.22	450	13.38	440	15.88	2,389
3	Egg	982	9.97	327	9.72	329	11.88	1,638
4	Fried food	848	8.61	326	9.69	221	7.98	1,395
5	Meat	1,325	13.46	449	13.35	321	11.59	2,095
6	Noodles/Pasta	438	4.45	147	4.37	147	5.31	732
7	Rice	279	2.83	88	2.62	90	3.25	457
8	Seafood	855	8.68	347	10.32	203	7.33	1,405
9	Soup	1,497	15.20	500	14.86	384	13.86	2,381
10	Vegetable/Fruit	707	7.18	227	6.75	220	7.94	1,154
	Total	9,846	100	3,364	100	2,770	100	15,980

1.4.4. Subset Selection Rules

The final experiments use all 15,980 Food-11 images retained after duplicate removal. Each retained image stays in its official training, validation, or evaluation partition. All 11 classes are included, with no further subsampling.

1.5. Data Preprocessing

1.5.1. Data Cleaning and Invalid Annotation Handling

Before resizing, each image is fully decoded using Pillow to verify readability. Unreadable files raise an error during this scan rather than being silently discarded. Duplicate removal follows the MD5 and pHash procedures described in the data leakage prevention section.

Image-level class annotations are obtained from the dataset folder names. A consistent class-to-index mapping is verified across the training, validation, and evaluation partitions. After cleaning, the retained class names and integer labels are checked for consistency, and all 11 classes are verified to remain present in each partition. No automatic relabelling is performed.

1.5.2. Image Resizing, Padding, and Normalization

All images are converted to RGB and resized using bilinear interpolation so that their longer dimension becomes 256 pixels while preserving the original aspect ratio. Images are downsampled or upsampled

as required. Each resized image is then centered on a black 256×256 canvas, with padding added along the shorter dimension when necessary as described in the Dataset Proposal. This produces a uniform input size without cropping the image content.

Pixel values are converted to floating-point tensors and scaled from $[0, 255]$ to $[0, 1]$. Each RGB channel is subsequently normalized as

$$\hat{x}_c = \frac{x_c - \mu}{\sigma},$$

using the fixed ImageNet mean $\mu = (0.485, 0.456, 0.406)$ and standard deviation $\sigma = (0.229, 0.224, 0.225)$. These statistics are used for all model configurations and are not estimated from the Food-11 partitions.

The resulting input tensor has shape $3 \times 256 \times 256$, corresponding to channels, height, and width. The same resizing, padding, and normalization rules are applied to the training, validation, and evaluation partitions.

1.5.3. Data Augmentation

Random horizontal flipping with probability 0.5 is applied to training images after resizing and padding. This introduces variation in image orientation while preserving the food category and requires no additional label modification. No rotation or other random augmentation is used.

Validation and evaluation images are processed without random augmentation. The same training augmentation policy is applied to all model configurations.

1.5.4. Input Tensor Shapes

Each preprocessed image has shape $(3, 256, 256)$, corresponding to RGB channels, height, and width. Pixel values are represented in 32-bit floating-point format and normalized using the ImageNet channel statistics.

Both SimpleCNN and ResNet-18 process the images using convolutional layers. The resulting feature maps are aggregated by global average pooling before being passed to the classification head.

1.5.5. Example Batch after Preprocessing

A full preprocessed training batch has the following expected shapes:

```
Images: torch.Size([128, 3, 256, 256])


Labels: torch.Size([128])
```

Training uses a batch size of 128 with shuffled sample order. Validation and evaluation also use a batch size of 128, without shuffling. Labels are integer class indices ranging from 0 to 10. The final batch may

contain fewer samples because incomplete batches are retained.

2. Methodology

2.1. Pipeline Overview

The classification pipeline follows a standard sequential flow across all experiments. The overall process is represented by the following diagram:

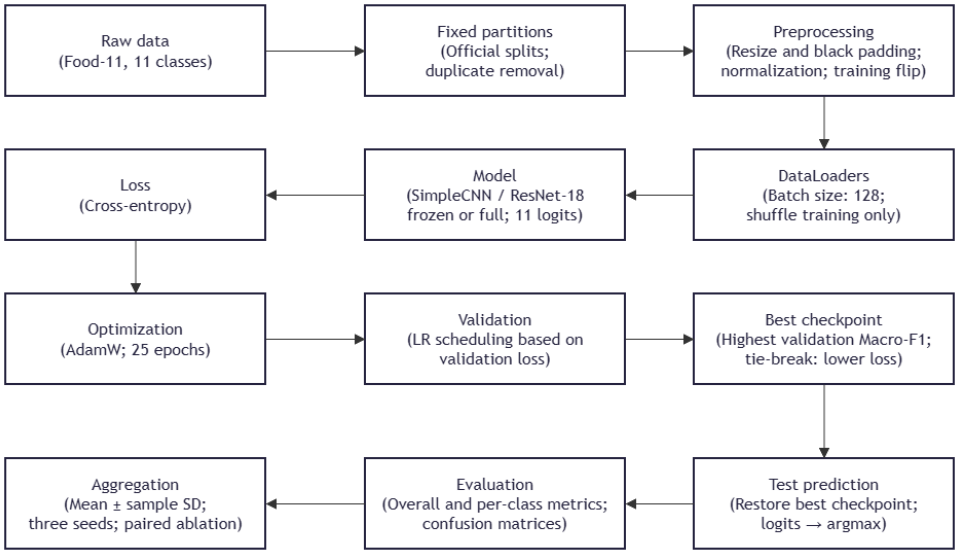

Figure 5: Pipeline Overview

Training and validation repeat until stopping. Each run restores its best checkpoint before test evaluation.

2.2. Convolutional Neural Network (CNN)

2.2.1. Architecture and input representation

The CNN uses a custom architecture (`SimpleCNN`) trained from scratch without a pretrained backbone. It consists of four convolutional blocks forming a feature extractor, followed by a classification head:

- **Feature Extraction Stage:**
  - **Block 1:** A 3×3 convolution maps 3 input channels to 16 output channels, followed by batch normalization, ReLU, and 2×2 max pooling. The spatial dimensions decrease from 256×256 to 128×128.
  - **Block 2:** A 3×3 convolution maps 16 channels to 32 channels, followed by batch normalization, ReLU, and 2×2 max pooling. The spatial dimensions decrease from 128×128 to 64×64.
  - **Block 3:** A 3×3 convolution maps 32 channels to 64 channels, followed by batch normalization, ReLU, and 2×2 max pooling. The spatial dimensions decrease from 64×64 to 32×32.

- **Block 4:** A 3×3 convolution maps 64 channels to 128 channels, followed by batch normalization, ReLU, and 2×2 max pooling. The spatial dimensions decrease from 32×32 to 16×16.

- **Classification Stage:** Adaptive global average pooling reduces the final feature maps to one value per channel. Flattening produces a 128-dimensional feature vector, followed by dropout with probability 0.3 and a single 128-to-11 linear layer. The output contains 11 logits corresponding to the Food-11 categories. No hidden fully connected layer is used.

The input is a tensor of preprocessed RGB images with shape $B \times 3 \times 256 \times 256$, where B is the number of images. Spatial dimensions are retained throughout the feature extractor. All convolutions use stride 1 and padding 1, while all max-pooling layers use stride 2. Dropout is applied only in the classification head.

2.2.2. Parameter Budget Analysis

SimpleCNN contains exactly 99,339 trainable parameters. This compact architecture serves as a from-scratch reference for comparison with the pretrained ResNet-18 configurations. Table 9 presents the layer-by-layer parameter breakdown.

For a convolutional layer, the parameter count includes the kernel weights and one bias per output channel. Each batch normalization layer contributes two trainable parameters per channel: a scale parameter γ and a shift parameter β.

Table 9: Layer-by-layer trainable parameter breakdown of `SimpleCNN`.

Block / Layer	Configuration	Parameter Formula	Count
Block 1	Conv2d($3 \to 16,\ 3 \times 3$)	$(16 \times 3 \times 3 \times 3) + 16$	448
	BatchNorm2d(16)	2×16	32
Block 2	Conv2d($16 \to 32,\ 3 \times 3$)	$(32 \times 16 \times 3 \times 3) + 32$	4,640
	BatchNorm2d(32)	2×32	64
Block 3	Conv2d($32 \to 64,\ 3 \times 3$)	$(64 \times 32 \times 3 \times 3) + 64$	18,496
	BatchNorm2d(64)	2×64	128
Block 4	Conv2d($64 \to 128,\ 3 \times 3$)	$(128 \times 64 \times 3 \times 3) + 128$	73,856
	BatchNorm2d(128)	2×128	256
Classifier	Linear($128 \to 11$)	$(11 \times 128) + 11$	1,419
Total Trainable			**99,339**

ReLU, max pooling, adaptive global average pooling, flattening, and dropout introduce no trainable parameters.

2.2.3. Strengths and limitations

2.2.3.a Strengths

- **Local connectivity and weight sharing:** Each 3×3 convolutional filter operates on a local neighborhood and shares its weights across spatial positions in the input feature maps. This exploits the spatial structure of RGB food images and allows the same pattern detector to respond at different locations. For fixed kernel sizes and channel counts, the number of convolutional parameters is independent of the input's spatial dimensions.
- **Hierarchical feature learning:** The four convolutional blocks can progressively combine local patterns into more complex visual representations. Earlier layers can capture edges, color transitions, and textures, while deeper layers can combine these responses into food-related visual patterns. This provides a mechanism for learning features directly from the Food-11 training images.
- **Reduced sensitivity to small positional changes:** Weight sharing allows filters to detect patterns at different image locations. Local max pooling retains the strongest response within each 2×2 region, which can reduce sensitivity to some small positional changes. However, these operations do not guarantee complete translation invariance.
- **Parameter efficiency:** Convolutional filters share weights across spatial locations, so larger feature maps require no additional filter weights. Global average pooling reduces each feature map to a single value, keeping the classifier's input size tied to the number of channels and avoiding a large fully connected layer.

2.2.3.b Limitations

- **High computational and memory cost** A small parameter count does not necessarily imply low computational or memory requirements. Convolutional filters are applied across many spatial positions, and intermediate activations must be retained for backpropagation. Consequently, computational cost and activation memory depend on image resolution, channel widths, and batch size, in addition to the number of parameters.
- **Limited spatial representation:** The convolution and pooling operations give each position in the final feature map a theoretical receptive field of 46×46 pixels. Meanwhile, four max-pooling operations reduce the spatial dimensions from 256×256 to 16×16, potentially discarding fine textures, ingredient details, and precise boundaries. Global average pooling aggregates responses across the entire feature map but removes explicit spatial positions, limiting the representation of

detailed relationships between distant image regions.

- **Limited built-in invariance:** Only translation is handled structurally; robustness to rotation, scale, or flipping is not guaranteed by the architecture and must be learned from data, for example through augmentation.
- **Low interpretability:** Unlike the linear baseline, the learned filters and feature maps do not map directly to pixel-level evidence for a class, so the model's predictions are harder to explain. Methods such as CAM or Grad-CAM can help explain individual predictions by highlighting regions that contribute to the predicted class score.
- **Limited rotation and scale invariance:** Although convolution and local pooling can reduce sensitivity to some small positional changes, the architecture does not guarantee invariance to rotation or scale changes.

2.2.4. Self-implemented components and library dependencies

Self-implemented: Complete custom ConvNet architecture (`SimpleCNN`, no pretrained backbone).
Library components: Convolution, ReLU activation, batch normalization, pooling, dropout, adaptive global average pooling, flattening, and linear layers from PyTorch.

2.3. Residual Network (Resnet-18)

2.3.1. Residual Learning and Input Representation

Residual learning introduces shortcut connections that combine a block's input with its learned transformation. For the basic residual block used in ResNet-18, the output is expressed as

$$\mathbf{y} = \mathrm{ReLU}\left(\mathcal{F}(\mathbf{x}) + S(\mathbf{x})\right), \tag{3}$$

where $\mathbf{x}$ is the block input, $\mathcal{F}(\mathbf{x})$ is the residual transformation, and $S(\mathbf{x})$ is the shortcut connection.

When the input and output dimensions match, $S(\mathbf{x}) = \mathbf{x}$. When the spatial dimensions or channel count change, the shortcut uses a 1×1 convolution followed by batch normalization to obtain compatible dimensions. The addition allows the block to learn a residual correction to the shortcut representation and provides an additional path for gradient propagation, helping the optimization of deeper networks.

Each input image is represented as a normalized RGB tensor with shape $3 \times 256 \times 256$. The same aspect-ratio-preserving resizing, black padding, and ImageNet normalization used for SimpleCNN are applied to ResNet-18. Spatial dimensions are retained during convolutional feature extraction.

2.3.2. ResNet-18

The model uses TorchVision's ResNet-18 implementation, initialized with ImageNet-1K pretrained weights (`IMAGENET1K_V1`). Its backbone consists of a convolutional stem followed by four residual stages containing two basic blocks each, giving eight residual blocks in total. The stage channel widths are 64, 128, 256, and 512.

The stem first applies a 7×7 convolution with stride 2 and padding 3, followed by batch normalization and ReLU. A 3×3 max-pooling layer with stride 2 and padding 1 then halves the spatial resolution again.

Each basic block has a residual branch and a shortcut branch. The residual branch applies two 3×3 convolutions with padding 1. Each convolution is followed by batch normalization, and ReLU is applied after the first normalization. The block adds the two branches element-wise and applies ReLU to the sum.

Downsampling occurs in the first block of Stages 2, 3, and 4. In these blocks, the first 3×3 convolution of the residual branch uses stride 2, while the second uses stride 1. The shortcut applies a 1×1 convolution with stride 2 followed by batch normalization. This projection matches the spatial dimensions and channel count of the residual branch before addition. All other blocks use stride 1 in both convolutions and an identity shortcut.

Table 10 summarizes the feature dimensions for a 256×256 input.

Table 10: ResNet-18 backbone dimensions for the configured input resolution.

Component	**Residual Blocks**	**Channels**	**Spatial Size**
Stem convolution	—	64	128×128
Max pooling	—	64	64×64
Stage 1	2	64	64×64
Stage 2	2	128	32×32
Stage 3	2	256	16×16
Stage 4	2	512	8×8
Global average pooling	—	512	1×1

Adaptive global average pooling summarizes the final feature maps into a 512-dimensional vector. The original 1000-class classifier is replaced with dropout at probability 0.3, followed by a single 512-to-11 linear layer. No hidden fully connected layer is added. The resulting head outputs 11 logits corresponding to the Food-11 categories.

The same architecture is used for both the frozen-backbone and full fine-tuning configurations; the two variants differ in which parameters are updated during training.

2.3.3. Parameter Budget Analysis.

After replacing the original classifier with the 11-class head, ResNet-18 contains 11,182,155 parameters. The pretrained backbone accounts for 11,176,512 parameters, while the new classification head contains

$$(512 \times 11) + 11 = 5,643$$

parameters. Dropout introduces no trainable parameters.

In the frozen-backbone configuration, only the 5,643 classification-head parameters are updated. In full fine-tuning, all 11,182,155 parameters are trainable. Table 11 presents the parameter breakdown for both configurations.

Table 11: Parameter breakdown of the adapted ResNet-18. Stage counts include projection shortcuts where present.

Component	Total	Trainable Frozen	Trainable Full
Stem	9,536	0	9,536
Stage 1	147,968	0	147,968
Stage 2	525,568	0	525,568
Stage 3	2,099,712	0	2,099,712
Stage 4	8,393,728	0	8,393,728
Classifier	5,643	5,643	5,643
Total	**11,182,155**	**5,643**	**11,182,155**

2.3.4. Strengths and limitations

2.3.4.a Strengths

- **Residual connections support optimization:** Shortcut connections provide an additional path for information and gradient propagation. They allow residual blocks to learn corrections to their shortcut representations, helping the optimization of a deeper feature extractor.
- **Reuse of pretrained representations:** ImageNet pretraining supplies an existing visual representation that can be transferred to Food-11. The backbone therefore does not need to learn all visual features from the Food-11 training images alone.
- **Controlled backbone adaptation:** The frozen configuration evaluates the usefulness of the existing features by training only the classification head. Full fine-tuning allows the backbone to adapt to Food-11 using a lower learning rate than the new head. These variants permit comparison of feature reuse and backbone adaptation within the same architecture.

2.3.4.b Limitations

- **Larger computational and memory requirements:** ResNet-18 contains substantially more parameters than the SimpleCNN baseline. Freezing the backbone reduces gradient computation and optimizer storage, but the complete backbone must still perform a forward pass for every image. A small trainable parameter count therefore does not imply a similarly small inference cost.
- **Restricted adaptation with a frozen backbone:** The frozen variant cannot modify backbone parameters or batch normalization running statistics. Its performance depends on how well the existing ImageNet representation separates the Food-11 categories, since only the final linear decision boundaries can be learned.
- **Overfitting risk during full fine-tuning:** Updating the entire network introduces a much larger trainable parameter budget. The model can become overly specialized to the training images or modify useful pretrained features. The lower backbone learning rate, weight decay, and validation-based checkpoint selection help control these risks but do not guarantee generalization.
- **Spatial compression and padding influence:** The backbone reduces the spatial dimensions from 256×256 to 8×8 before global average pooling. This can discard fine ingredient details and precise spatial arrangements. Global average pooling also averages over all feature-map positions without explicitly excluding features associated with padded image regions.

2.3.5. Self-implemented components and library dependencies

Self-implemented Custom `FoodResNet` wrapper with an 11-class classifier and support for frozen-backbone training and full fine-tuning.

Library components ResNet-18 and ImageNet-1K pretrained weights from TorchVision. Dropout and linear layers from PyTorch.

2.4. Shared Loss and evaluation metrics

2.4.1. Loss function

All models use multiclass cross-entropy loss. For a batch of B images, each model outputs a logit matrix $\mathbf{Z} \in \mathbb{R}^{B\times 11}$, where each row contains eleven unnormalized class scores. For image i with ground-truth label y_i, the loss is

$$\ell_i = -\log\left(\frac{\exp(Z_{i,y_i})}{\sum_{c=0}^{10}\exp(Z_{i,c})}\right). \tag{4}$$

The batch loss is the mean over its samples:

$$\mathcal{L} = \frac{1}{B}\sum_{i=1}^{B}\ell_i. \tag{5}$$

2.4.2. Predictive evaluation metrics

Test Macro-F1-score is the primary metric for model comparison, giving equal weight to all 11 classes. Secondary metrics include top-1 accuracy, Macro-Precision, Macro-Recall, and test cross-entropy loss. Per-class metrics are also reported to examine performance across individual food categories.

Let $K = 11$ denote the number of Food-11 classes and N the number of images in the evaluated partition. For each image n, the predicted label is the class with the highest output logit:

$$\hat{y}_n = \arg \max_{c \in \{0,\ldots,K-1\}} z_{n,c}, \tag{6}$$

where $z_{n,c}$ is the logit for class c.

For each class c, a one-vs-rest interpretation treats class c as positive and all remaining classes as negative:

- TP_c (true positives): images whose ground true and predicted labels are both class c.
- FN_c (false negatives): images belonging to class c that are predicted as another class.
- FP_c (false positives): images belonging to another class that are predicted as class c.
- TN_c (true negatives): images whose ground true and predicted labels are both different from class c.

For each class, $TP_c + FN_c + FP_c + TN_c = N$. Predictive metrics are computed from predictions collected across the complete evaluated partition, rather than by averaging mini-batch metric values.

2.4.2.1 Primary metric: Macro-F1-score.

Macro-F1-score is the unweighted average of the individual class F1-scores:

$$\text{Macro-F1} = \frac{1}{K} \sum_{c=0}^{K-1} F1_c = \frac{1}{K} \sum_{c=0}^{K-1} \frac{2TP_c}{2TP_c + FP_c + FN_c}. \tag{7}$$

Each class contributes equally, regardless of its number of images. This is appropriate for Food-11, whose class supports are unequal, because performance on less frequent categories remains represented in the overall score.

Each class F1-score balances precision and recall, accounting for both false positives and false negatives. Macro-F1 is obtained by averaging class-wise F1-scores, not by taking the harmonic mean of Macro-Precision and Macro-Recall.

The checkpoint with the highest validation Macro-F1 is selected for final evaluation. If validation Macro-

F1 values are equal, the checkpoint with the lower validation cross-entropy loss is selected. Evaluation-set performance is not used for checkpoint selection.

2.4.2.2 Secondary predictive metrics.

Top-1 Accuracy: Top-1 accuracy measures the proportion of images whose predicted labels match their ground-truth labels:

$$\text{Accuracy} = \frac{\sum_{c=0}^{K-1} TP_c}{N}. \tag{8}$$

Each correctly classified image contributes to exactly one class's true-positive count. Accuracy summarizes overall classification correctness, but more frequent classes contribute more images to this measure.

Macro-Precision: Macro-Precision is the unweighted average of class-wise precision:

$$\text{Macro-Precision} = \frac{1}{K} \sum_{c=0}^{K-1} \frac{TP_c}{TP_c + FP_c}. \tag{9}$$

It summarizes the reliability of the model's class assignments while giving every class equal weight. False positives reduce precision.

Macro-Recall: Macro-Recall is the unweighted average of class-wise recall:

$$\text{Macro-Recall} = \frac{1}{K} \sum_{c=0}^{K-1} \frac{TP_c}{TP_c + FN_c}. \tag{10}$$

It measures how successfully the model recognizes images from each category, giving every class equal weight. False negatives reduce recall.

2.4.2.3 Per-class predictive metrics.

One-vs-rest accuracy, precision, recall, F1-score are reported separately for each food category to identify strengths and weaknesses hidden by the overall averages.

Per-class one-vs-rest accuracy:

This metric measures the proportion of images correctly identified as belonging or not belonging to class c:

$$\text{Accuracy}_c^{\text{OvR}} = \frac{TP_c + TN_c}{TP_c + TN_c + FP_c + FN_c} = \frac{TP_c + TN_c}{N}. \tag{11}$$

Unlike recall, this metric includes true negatives. An image contributes to TN_c whenever both its true and predicted labels differ from class c, even if it is misclassified between two other classes. Consequently, a high one-vs-rest accuracy does not necessarily indicate successful recognition of the target category.

Dessert is the largest class in the cleaned dataset, accounting for 14.95% of all retained images. In the final evaluation partition, it contains 440 of the 2,770 images, corresponding to 15.88%. A classifier that never predicts Dessert would therefore achieve

$$\text{Accuracy}_{\text{Dessert}}^{\text{OvR}} = \frac{2770 - 440}{2770} = 84.12\%, \tag{12}$$

despite having zero recall and zero F1-score for Dessert.

Even the largest category therefore has enough negative examples to produce a seemingly high one-vs-rest accuracy without recognizing any positive examples. This metric is interpreted alongside precision, recall, and F1-score; Macro-F1 remains the primary comparison metric. **Per-class precision:**

$$P_c = \frac{TP_c}{TP_c + FP_c}. \tag{13}$$

Precision measures the proportion of predictions assigned to class c that are correct. Low precision indicates that images from other categories are frequently assigned to this class.

Per-class recall:

$$R_c = \frac{TP_c}{TP_c + FN_c}. \tag{14}$$

Recall measures the proportion of images belonging to class c that are correctly recognized. Low recall indicates that many images of this class are assigned to other categories.

Per-class F1-score:

$$F1_c = \frac{2P_cR_c}{P_c + R_c} = \frac{2TP_c}{2TP_c + FP_c + FN_c}. \tag{15}$$

The F1-score combines class-specific precision and recall. A high score requires both reliable class assignments and successful recognition of images belonging to the class.

2.5. Training setup

To ensure fairness, reproducibility, and rigorous comparative analysis, an identical and strictly standardized training protocol was enforced across all models. Table 13 summarizes the exact hyperparameter configurations and experimental environment.

Table 12: Training and Optimization Hyperparameters for SimpleCNN and ResNet-18

Hyperparameter / Component	Specification / Value
Optimizer	`torch.optim.AdamW`, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, and weight decay $= 10^{-2}$
Loss Function	Unweighted `CrossEntropyLoss`
Initial Learning Rate	**SimpleCNN:** 10^{-3} for all parameters. **ResNet-18 frozen:** 10^{-3} for the classification head. **ResNet-18 full:** 10^{-4} for the backbone and 10^{-3} for the classification head.
Batching and Data Loading	Batch size 128 for training, validation, and evaluation. Training samples are shuffled; validation and evaluation samples are not. Data loading uses `num_workers=0`.
Training Duration	A fixed budget of 25 epochs for every configuration; no early stopping
Learning Rate Scheduler	`ReduceLROnPlateau`, `mode=min`, factor $= 0.5$, patience $= 3$, and minimum learning rate $= 10^{-6}$ for each parameter group. The default threshold is 10^{-4} with `threshold_mode=rel`. The scheduler is stepped on validation loss.
Input and Normalization	RGB inputs of shape $3 \times 256 \times 256$. Aspect-ratio-preserving bilinear resizing and centered black padding are applied. Pixel values are scaled to $[0,1]$ and normalized using ImageNet mean $(0.485, 0.456, 0.406)$ and standard deviation $(0.229, 0.224, 0.225)$.
Data Augmentation	Training only: random horizontal flipping with probability 0.5. No random augmentation is applied to validation or evaluation images.
Regularization	**SimpleCNN:** Batch normalization in all four convolutional blocks and dropout with probability 0.3 in the classification head. **ResNet-18:** Backbone batch normalization and dropout with probability 0.3 in the new head. Frozen-backbone batch normalization remains in evaluation mode.
Weight Initialization	**SimpleCNN:** Trained from scratch. **ResNet-18:** ImageNet-1K pretrained backbone (`IMAGENET1K_V1`) with a newly initialized 11-class classification head.
Checkpoint Criterion	Highest validation Macro-F1-score. Equal scores are resolved using lower validation loss. The selected checkpoint is restored before final evaluation.

Table 13: Training and Optimization Hyperparameters for SimpleCNN and ResNet-18

Hyperparameter / Component	Specification / Value
Seeds	Training seeds 42, 43, and 44 for each configuration. The cleaned Food-11 partitions are derived from the official splits and held fixed across all runs.
Hardware & Environment	NVIDIA Tesla T4 GPU, Google Colab, Linux, Python 3.13.16, PyTorch 2.11.0 - 2.14.1+cu130, TorchVision 0.26.0 - 0.29.1 +cu130, CUDA 13.0, NumPy 2.4.6, pandas 2.2.3, and scikit-learn 1.6.1
Numerical Precision	FP32 (`torch.float32`); no mixed-precision training is used
Reported Training Time	Sum of the per-epoch training and validation timings. Includes batch loading, preprocessing, training augmentation, device transfer, forward and backward passes, optimizer updates, and metric computation. Excludes dataset preparation, model initialization, checkpoint and CSV writes, printed logging, scheduler updates, and final test evaluation.
Hyperparameter Configuration	Hyperparameters were selected manually and held fixed across training seeds for each configuration.

2.6. Experimental Design

Two experiments are defined: a comparison of the three model configurations and a controlled ablation of training-time horizontal flipping. The model comparison includes SimpleCNN trained from scratch, ResNet-18 with a frozen backbone, and ResNet-18 with full fine-tuning.

2.6.1. Experiment 1: Model Configuration Comparison

2.6.1.1 Hypothesis.

The pretrained ResNet-18 configurations are expected to outperform the from-scratch SimpleCNN baseline in test Macro-F1. Within ResNet-18, full fine-tuning is expected to improve performance over a frozen backbone by allowing the pretrained representations to adapt to Food-11.

2.6.1.2 Changed factor.

The model configuration is varied among SimpleCNN, ResNet-18 with a frozen backbone, and ResNet-18 with full fine-tuning. SimpleCNN and ResNet-18 differ in architecture, initialization, and total parameter count. Within ResNet-18, the configuration determines which parameters are trainable. The frozen variant trains only the classifier and preserves the backbone's parameters and BatchNorm running statistics. Full fine-tuning trains both the backbone and the classifier and updates the backbone's BatchNorm running statistics. Learning rates follow the configuration-specific settings stated above.

2.6.1.3 Fixed factors.

All three configurations use the same cleaned data partitions, input preprocessing, horizontal-flip augmentation, batch size, optimizer algorithm, weight decay, training budget, learning-rate scheduler policy, and checkpoint-selection criterion. Comparisons are paired by training seed.

For each matched seed, the two ResNet-18 variants share the same architecture, total parameter count, ImageNet-1K pretrained backbone weights, and initial classifier weights. Both use a classifier with dropout probability 0.3 followed by a linear layer with 512 inputs and 11 outputs.

2.6.1.4 Metrics.

Test Macro-F1 is compared across all three configurations, supported by the secondary predictive metrics and computational costs. Training and validation curves support the analysis of convergence, underfitting and overfitting.

2.6.1.5 Conclusion criteria.

The comparison considers mean test Macro-F1, variability across seeds, and computational cost. The SimpleCNN-versus-ResNet comparison evaluates complete configurations and cannot attribute performance differences solely to pretraining, because architecture and capacity also differ.

2.6.2. Experiment 2: Horizontal-flip Augmentation Ablation

2.6.2.1 Hypothesis.

Horizontal flipping during training is expected to improve generalization and increase test Macro-F1 compared with training without augmentation.

2.6.2.2 Changed factor.

Training-time horizontal flipping is enabled with probability 0.5 or disabled with probability 0. This isolates the contribution of the horizontal-flip operation used in the standard training pipeline.

2.6.2.3 Fixed factors.

The augmentation-enabled and augmentation-disabled conditions are compared separately for SimpleCNN, ResNet-18 with a frozen backbone, and ResNet-18 with full fine-tuning, with all other training settings held fixed within each configuration.

2.6.2.4 Metrics.

Test Macro-F1 is the primary metric. Accuracy, per-class metrics, and training and validation curves provide supplementary evidence. For each configuration c and training seed s, the paired difference is calculated as

$$\Delta_{c,s}^{\mathrm{Aug}} = \mathrm{MacroF1}_{c,s}^{\mathrm{aug}} - \mathrm{MacroF1}_{c,s}^{\mathrm{noaug}}. \tag{16}$$

These differences are summarized as mean $\pm$ sample standard deviation across seeds 42, 43, and 44.

2.6.2.5 Conclusion criteria.

A positive mean difference favors horizontal-flip augmentation, whereas a negative mean difference favors training without augmentation. The standard deviation describes the variability of the augmentation effect across seeds. A mean difference that is small compared with its standard deviation is interpreted cautiously as inconclusive evidence of improvement.

3. Implementation Results

3.1. Training Behavior

Training metrics are aggregated from batch predictions made throughout each epoch while model parameters are updated after every batch. Validation metrics are computed afterward using the fixed parameters obtained at the end of that epoch. In the main experiments, training uses augmented images, whereas validation uses unaugmented images. Dropout and BatchNorm can also behave differently in training and evaluation modes. These differences can explain why validation loss is sometimes lower or validation accuracy is higher.

All runs complete 25 epochs. The checkpoint used for test evaluation is selected by the highest validation Macro-F1, with lower validation cross-entropy used to break ties. Reported training times include training and validation and exclude checkpoint and CSV writing. Timing results are summarized across seeds 42, 43, and 44 using mean $\pm$ sample standard deviation.

3.1.1. Baseline (SimpleCNN)

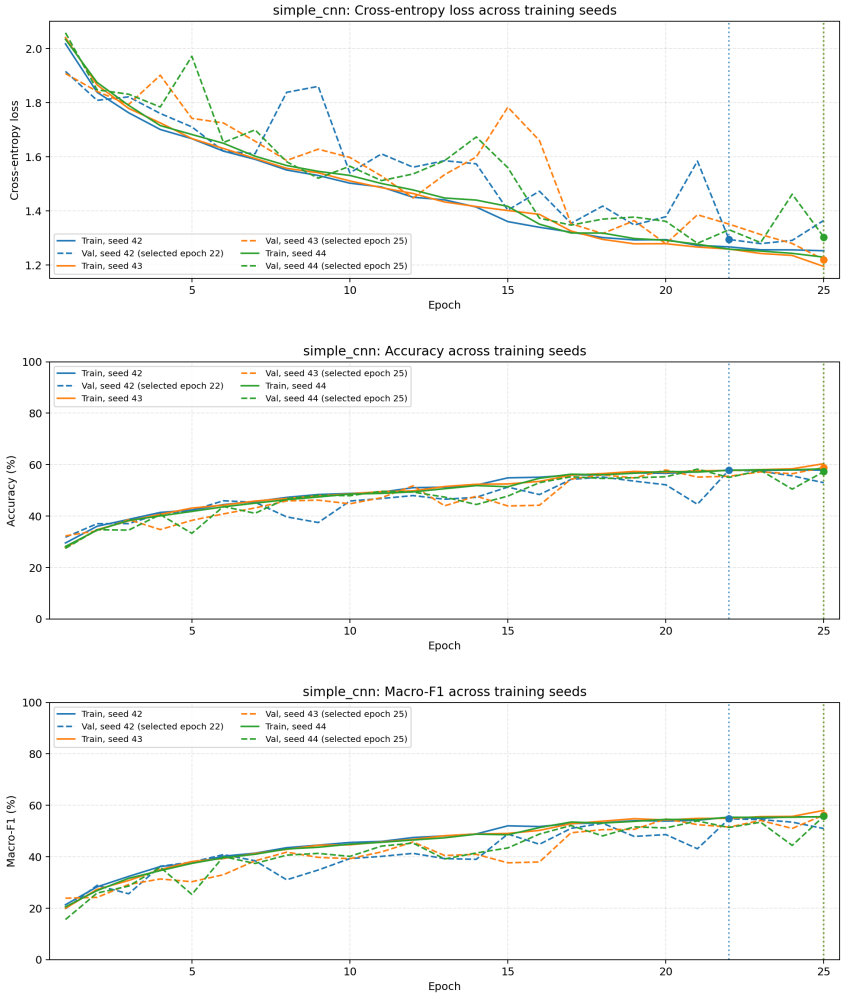

Figure 6: Training and validation loss, accuracy, and Macro-F1 of SimpleCNN across seeds 42, 43, and 44. Dotted lines and markers identify the validation-selected checkpoints.

Figure 6 shows gradual learning across all three seeds. Training cross-entropy decreases from approximately 2.02–2.04 to 1.19–1.25, while training accuracy increases from approximately 28–30% to 58–60%. Validation loss decreases overall, with temporary increases accompanied by drops in validation accuracy and Macro-F1.

The selected checkpoints occur at epochs 22, 25, and 25 for seeds 42, 43, and 44, respectively. Their validation Macro-F1 scores are 54.74%, 56.22%, and 55.74%. For seed 42, validation loss is lowest at epoch 23, while validation Macro-F1 is highest at the selected epoch 22.

Training and validation performance remain relatively low and track each other closely. This pattern suggests incomplete fitting within the 25-epoch budget. Training loss is still improving late in training, and two seeds select the final epoch. The runs leave open how much of the remaining limitation comes from model capacity, optimization, or training duration. Mean training time is 118.78 ± 4.49 seconds per

epoch.

3.1.2. frozen-backbone ResNet-18

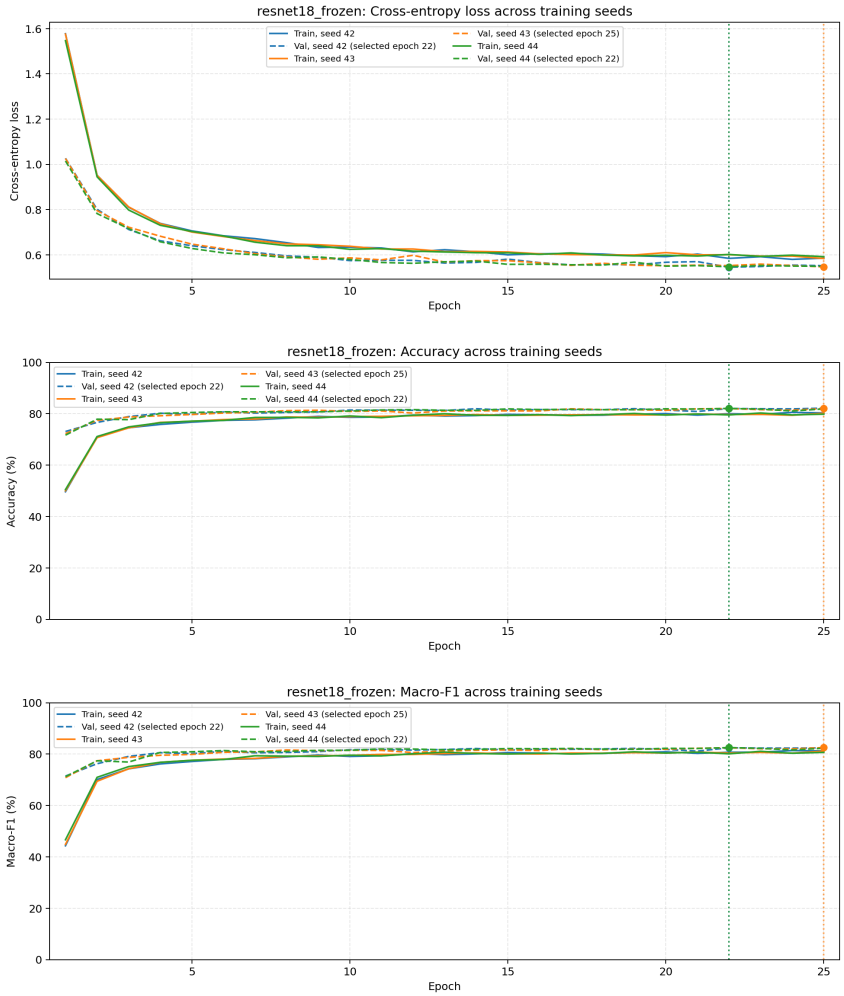

Figure 7: Training and validation learning curves of frozen-backbone ResNet-18 across three seeds. Each seed's selected epoch is shown in the legend and marked on the curves.

Figure 7 shows rapid initial improvement followed by progressively smaller gains. Training loss decreases from approximately 1.55–1.58 to 0.58–0.59, while validation loss decreases from approximately 1.01–1.03 to about 0.55. Validation accuracy and Macro-F1 stabilize during the later epochs.

The selected checkpoints occur at epochs 22, 25, and 22 for seeds 42, 43, and 44, respectively. Their validation Macro-F1 scores are 82.47%, 82.54%, and 82.54%. Validation performance remains slightly higher than training performance, which is compatible with training-time augmentation and head dropout. The backbone stays in evaluation mode during training, so its BatchNorm running statistics remain fixed.

Validation performance is broadly stable in the later epochs, suggesting limited additional gains from head-only training under the current protocol. Mean training time is 118.81 ± 3.89 seconds per epoch. Each training image still requires a forward pass through the complete backbone.

3.1.3. ResNet-18 Full Fine-Tuning

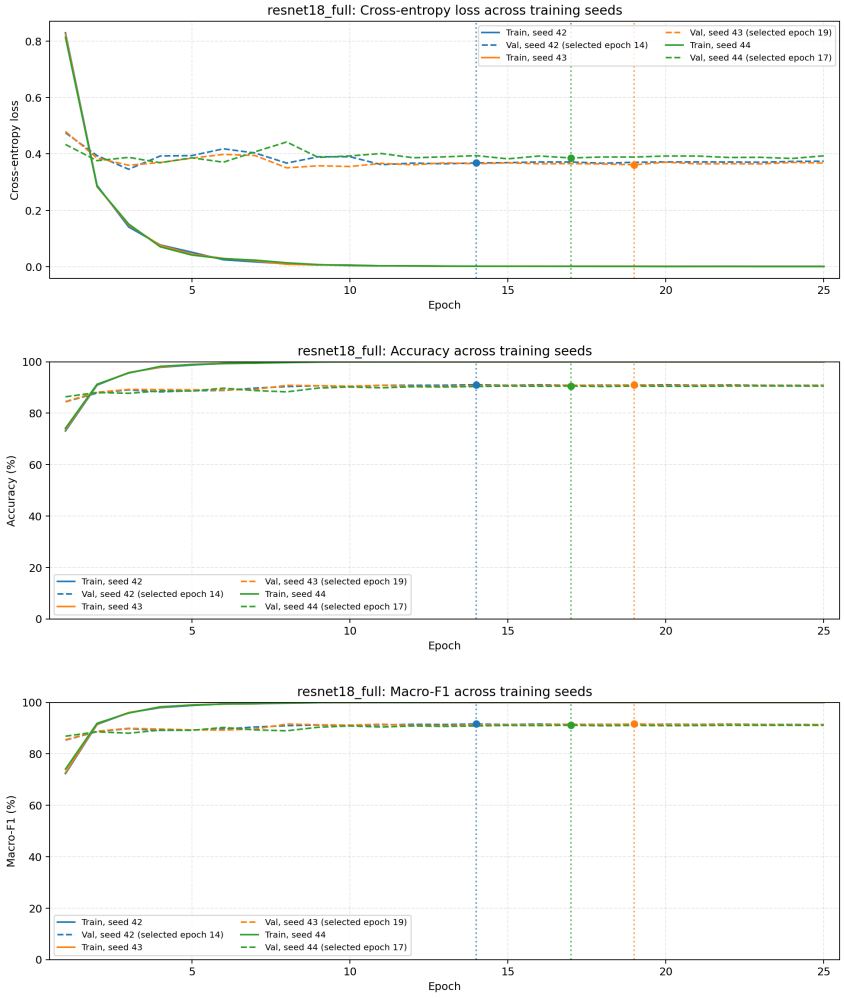

Figure 8: Training and validation learning curves of fully fine-tuned ResNet-18 across three seeds. Selected checkpoints are marked separately for each seed.

Figure 8 shows faster fitting of the training data. Training loss decreases from approximately 0.81–0.83 to about 0.001, and the logged training accuracy reaches 100% by epochs 11–13. Validation accuracy and Macro-F1 improve rapidly at first and then remain near 91%.

The selected checkpoints occur at epochs 14, 19, and 17 for seeds 42, 43, and 44, respectively. Their validation Macro-F1 scores are 91.62%, 91.65%, and 91.12%. The minimum validation-loss epochs are 3, 8, and 4, respectively.

After its early minimum, validation loss broadly plateaus or rises while training loss continues to fall. This persistent gap is consistent with overfitting later in training, despite strong validation performance. Mean training time is 144.78 ± 2.89 seconds per epoch. Full fine-tuning takes longer than linear probing because the optimizer updates the backbone as well as the classification head.

3.2. Quantitative Results

3.2.1. Overall Performance and Computational Cost

Table 14 summarizes predictive performance and computational cost under the training settings described in the Methodology section. Results are reported as mean ± sample standard deviation across three training seeds.

Model size and computational cost

Model	Total parameters	Trainable parameters	Epochs	Total train time per run (s)	Train time per epoch (s)	Inference (ms/sample)
SimpleCNN	99,339	99,339	25	$2{,}969.5 \pm 112.3$	118.78 ± 4.49	0.8054 ± 0.1036
ResNet-18 (frozen)	11,182,155	5,643	25	$2{,}970.3 \pm 97.2$	118.81 ± 3.89	2.6574 ± 0.4641
ResNet-18 (full)	11,182,155	11,182,155	25	$3{,}619.5 \pm 72.1$	144.78 ± 2.89	2.4819 ± 0.1453

Predictive performance

Model	Top-1 Accuracy (%)	Macro-Precision (%)	Macro-Recall (%)	Macro-F1 (%)	Test CE loss
SimpleCNN	60.40 ± 1.02	63.48 ± 2.25	57.23 ± 0.51	57.58 ± 0.84	1.2060 ± 0.0273
ResNet-18 (frozen)	84.62 ± 0.24	85.45 ± 0.10	85.23 ± 0.19	85.26 ± 0.17	0.4938 ± 0.0036
ResNet-18 (full)	$\mathbf{91.37} \pm 0.43$	$\mathbf{91.65} \pm 0.37$	$\mathbf{91.74} \pm 0.40$	$\mathbf{91.67} \pm 0.39$	$\mathbf{0.3197} \pm 0.0148$

Table 14: Predictive performance, model size, and computational cost of SimpleCNN and the two ResNet-18 configurations on Food-11. Performance and timing results are reported as mean ± sample standard deviation across training seeds 42, 43, and 44. Macro-F1 is the primary predictive metric. Bold values indicate the best mean predictive results.

Fully fine-tuned ResNet-18 achieves the highest mean test Macro-F1 at $91.67 \pm 0.39\%$, followed by frozen ResNet-18 at $85.26 \pm 0.17\%$ and SimpleCNN at $57.58 \pm 0.84\%$. The two ResNet-18 configurations exceed SimpleCNN by 27.68 and 34.09 %, respectively. Accuracy, Macro-Precision, and Macro-Recall follow the same ordering. Test cross-entropy also improves from 1.2060 for SimpleCNN to 0.4938 for the frozen configuration and 0.3197 for full fine-tuning.

The frozen configuration has the smallest Macro-F1 standard deviation across the three seeds. SimpleCNN shows greater variation in both Macro-F1 and Macro-Precision across the observed runs.

SimpleCNN contains 99,339 parameters, compared with 11,182,155 for either ResNet-18 variant. Freezing ResNet-18 reduces the trainable count to 5,643, while its training time remains close to that of SimpleCNN at 2,970.3 versus 2,969.5 seconds. The frozen backbone still performs a forward pass for every image. Full fine-tuning takes 3,619.5 seconds on average, approximately 21.9% longer than the frozen configuration.

SimpleCNN has the lowest measured inference time at 0.8054 ± 0.1036 milliseconds per image. Both

ResNet-18 variants take approximately 2.48–2.65 milliseconds per image. Their similar inference times reflect the shared forward architecture.

3.2.2. Per-Class Performance

Table 15: Per-class test metrics of SimpleCNN on Food-11, reported as mean ± sample standard deviation across training seeds 42, 43, and 44. Bold and underlined values indicate the highest and lowest means, respectively, within each metric column.

Class	OvR Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
Bread	89.64 ± 2.19	48.92 ± 8.70	47.51 ± 11.44	46.87 ± 4.00
Dairy product	95.38 ± 0.07	76.47 ± 3.09	$\underline{18.82} \pm 2.75$	$\underline{30.10} \pm 3.44$
Dessert	$\underline{83.25} \pm 1.52$	$\underline{48.39} \pm 3.52$	66.67 ± 6.43	55.83 ± 0.99
Egg	90.29 ± 0.58	61.41 ± 4.42	50.46 ± 3.70	55.22 ± 1.43
Fried food	92.07 ± 1.39	51.22 ± 6.90	64.71 ± 5.56	56.74 ± 2.42
Meat	92.77 ± 0.33	74.98 ± 8.61	59.61 ± 15.22	64.87 ± 7.50
Noodles-Pasta	95.85 ± 0.85	62.09 ± 9.56	61.68 ± 3.49	61.44 ± 3.66
Rice	96.57 ± 0.66	50.42 ± 15.44	39.63 ± 6.70	43.00 ± 2.99
Seafood	94.33 ± 0.32	65.91 ± 10.97	54.52 ± 14.53	57.81 ± 5.87
Soup	92.80 ± 1.58	75.20 ± 9.87	74.57 ± 5.36	74.37 ± 2.88
Vegetable-Fruit	$\mathbf{97.85} \pm 0.30$	$\mathbf{83.26} \pm 3.21$	$\mathbf{91.36} \pm 0.45$	$\mathbf{87.10} \pm 1.55$

Vegetable-Fruit has the highest mean F1-score for SimpleCNN at 87.10%. Dairy product has the lowest at 30.10%. Its precision is 76.47%, while its recall is only 18.82%, indicating that the model misses most true examples of the class. Correctly rejected images from other classes contribute to its 95.38% OvR accuracy.

Table 16: Per-class test metrics of ResNet-18 with a frozen backbone on Food-11, reported as mean ± sample standard deviation across training seeds 42, 43, and 44. Bold and underlined values indicate the highest and lowest means, respectively, within each metric column.

Class	OvR Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
Bread	95.28 ± 0.15	$\underline{75.66} \pm 2.39$	75.75 ± 2.45	75.65 ± 0.24
Dairy product	97.57 ± 0.09	81.64 ± 2.73	$\underline{70.07} \pm 1.80$	$\underline{75.37} \pm 0.41$
Dessert	$\underline{93.39} \pm 0.06$	80.12 ± 0.93	77.73 ± 2.02	78.89 ± 0.58
Egg	95.09 ± 0.10	79.90 ± 1.12	78.42 ± 2.19	79.13 ± 0.70
Fried food	97.20 ± 0.15	82.07 ± 2.25	83.11 ± 2.23	82.55 ± 0.75
Meat	96.79 ± 0.23	83.56 ± 1.90	90.03 ± 0.62	86.66 ± 0.74
Noodles-Pasta	$\mathbf{99.70} \pm 0.02$	$\mathbf{96.64} \pm 0.01$	$\mathbf{97.73} \pm 0.39$	$\mathbf{97.18} \pm 0.20$
Rice	99.49 ± 0.04	90.43 ± 0.92	94.44 ± 1.11	92.39 ± 0.55
Seafood	97.45 ± 0.13	82.41 ± 1.77	82.92 ± 0.57	82.66 ± 0.63
Soup	98.33 ± 0.08	92.90 ± 0.82	95.23 ± 0.66	94.04 ± 0.28
Vegetable-Fruit	98.95 ± 0.06	94.59 ± 1.47	92.12 ± 1.84	93.32 ± 0.45

Frozen ResNet-18 improves mean F1-score over SimpleCNN in all 11 classes. The largest gains occur for Rice and Dairy product at 49.39 and 45.27 %, respectively. Noodles-Pasta has the highest mean F1-score at 97.18%. Dairy product remains the lowest-scoring class at 75.37%, closely followed by Bread at 75.65%.

Table 17: Per-class test metrics of ResNet-18 with full fine-tuning on Food-11, reported as mean ± sample standard deviation across training seeds 42, 43, and 44. Bold and underlined values indicate the highest and lowest means, respectively, within each metric column.

Class	OvR Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
Bread	97.36 ± 0.23	87.37 ± 1.13	85.07 ± 2.33	86.19 ± 1.30
Dairy product	98.30 ± 0.07	$\underline{86.59} \pm 0.36$	$\underline{80.50} \pm 1.42$	$\underline{83.43} \pm 0.83$
Dessert	$\underline{96.04} \pm 0.12$	87.51 ± 0.69	87.58 ± 0.47	87.54 ± 0.33
Egg	97.91 ± 0.25	91.14 ± 2.11	91.29 ± 0.46	91.20 ± 0.96
Fried food	98.23 ± 0.07	88.63 ± 0.66	89.29 ± 1.71	88.95 ± 0.58
Meat	97.79 ± 0.24	89.47 ± 1.29	91.69 ± 0.72	90.57 ± 0.99
Noodles-Pasta	$\mathbf{99.89} \pm 0.04$	$\mathbf{99.10} \pm 0.78$	98.87 ± 0.39	$\mathbf{98.98} \pm 0.34$
Rice	99.72 ± 0.02	93.03 ± 0.56	$\mathbf{98.89} \pm 0.00$	95.87 ± 0.30
Seafood	98.68 ± 0.08	90.71 ± 0.73	91.30 ± 1.14	91.00 ± 0.54
Soup	99.23 ± 0.08	97.30 ± 0.15	97.14 ± 0.52	97.22 ± 0.28
Vegetable-Fruit	99.59 ± 0.06	97.28 ± 0.45	97.58 ± 0.26	97.43 ± 0.34

Full fine-tuning further improves mean F1-score in every class. The largest gains over the frozen con-

figuration are 12.07 % for Egg and 10.54 for Bread. Noodles-Pasta remains the highest-scoring class at $98.98 \pm 0.34\%$. Dairy product improves to $83.43 \pm 0.83\%$ but retains the lowest mean F1-score and recall. Its mean recall of 80.50% indicates that roughly one in five Dairy product images is assigned to another class.

Cross-model summary: Vegetable-Fruit, Soup, and Noodles-Pasta rank among the four highest-scoring classes by mean F1-score across all three configurations. Dairy product has the lowest mean F1-score and recall in every configuration, while Bread consistently ranks among the bottom three classes by mean F1-score. Rice ranks second-lowest for SimpleCNN and fourth-highest for both ResNet-18 configurations, showing that its relative difficulty depends strongly on the model.

3.2.3. Test Confusion Matrices

Figures 9–11 show test confusion matrices across seeds 42, 43, and 44. Rows represent true classes and columns represent predicted classes. The left panels show image counts and the right panels show row-normalized percentages. Each cell reports the mean and sample standard deviation across seeds. The diagonal of the normalized matrix gives class recall.

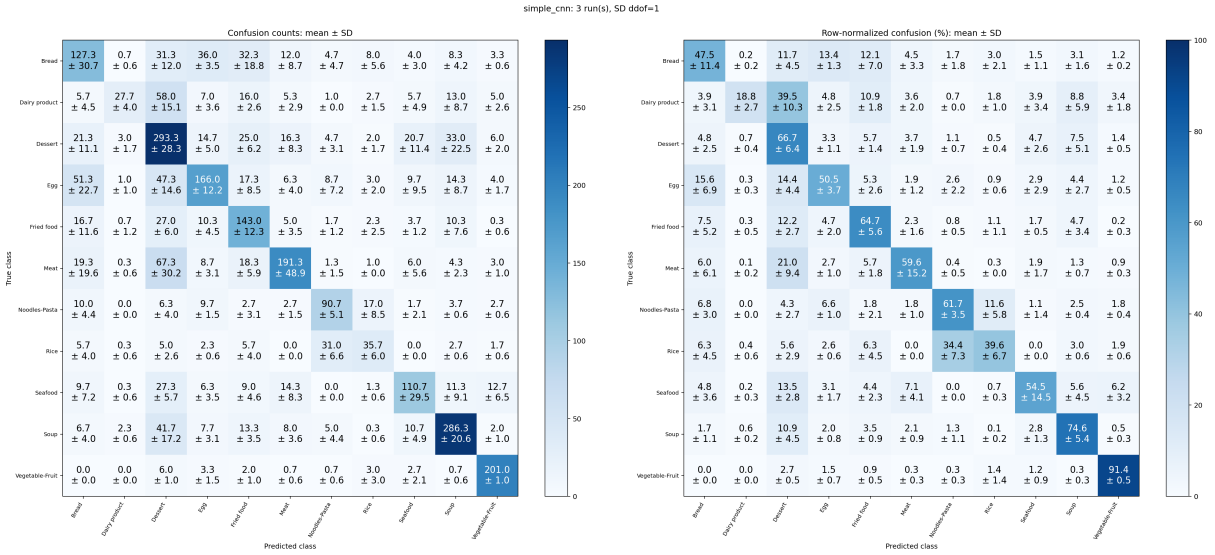

Figure 9: Food-11 test confusion matrices for SimpleCNN.

SimpleCNN often predicts Dessert for images from other classes. For Dairy product, $39.5 \pm 10.3\%$ of images are classified as Dessert and $18.8 \pm 2.7\%$ are classified correctly. Another prominent error is Rice predicted as Noodles-Pasta, which affects $34.4 \pm 7.3\%$ of Rice images. Vegetable-Fruit has the highest recall at $91.4 \pm 0.5\%$.

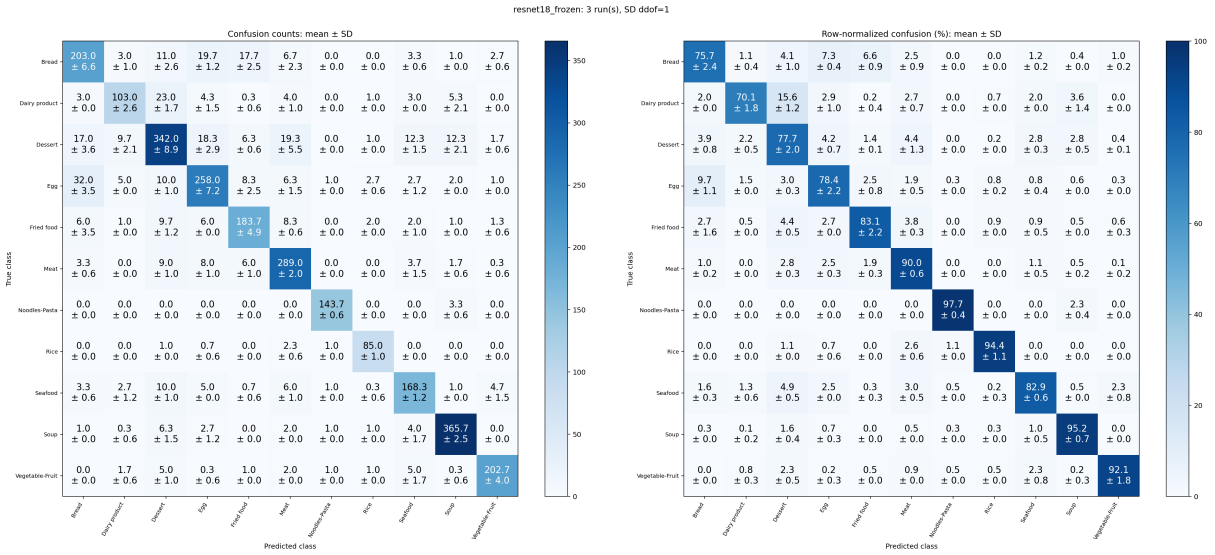

Figure 10: Food-11 test confusion matrices for ResNet-18 with a frozen backbone.

With frozen ResNet-18, Rice recall rises from 39.6% to 94.4%, and the share of Rice images predicted as Noodles-Pasta falls from 34.4% to 1.1%. Dairy product recall improves to 70.1%. Dessert remains its most frequent incorrect prediction at $15.6 \pm 1.2\%$.

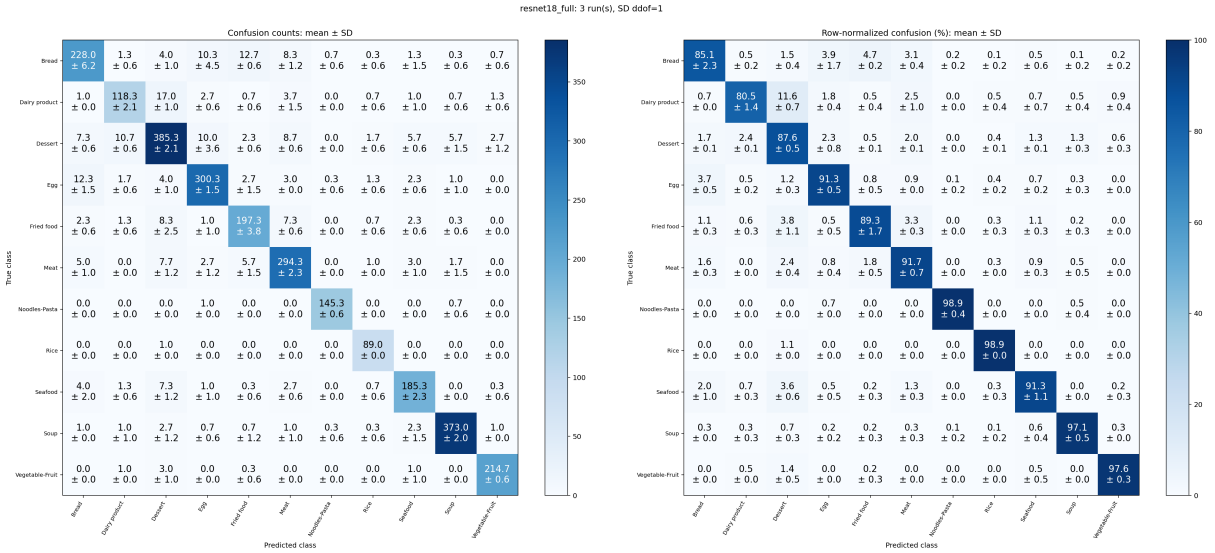

Figure 11: Food-11 test confusion matrices for ResNet-18 with full fine-tuning.

Full fine-tuning increases mean recall for all 11 classes relative to the frozen configuration. The proportion of Egg images predicted as Bread decreases from 9.7% to 3.7%. Dairy product retains the lowest recall at $80.5 \pm 1.4\%$. Its confusion with Dessert is asymmetric. The model predicts Dessert for 11.6% of Dairy product images and Dairy product for 2.4% of Dessert images.

3.3. Qualitative Results

The examples come from the Food-11 test set and use each model's validation-selected checkpoint for training seed 42. Examples are selected separately for each model.

3.3.1. Correct Predictions

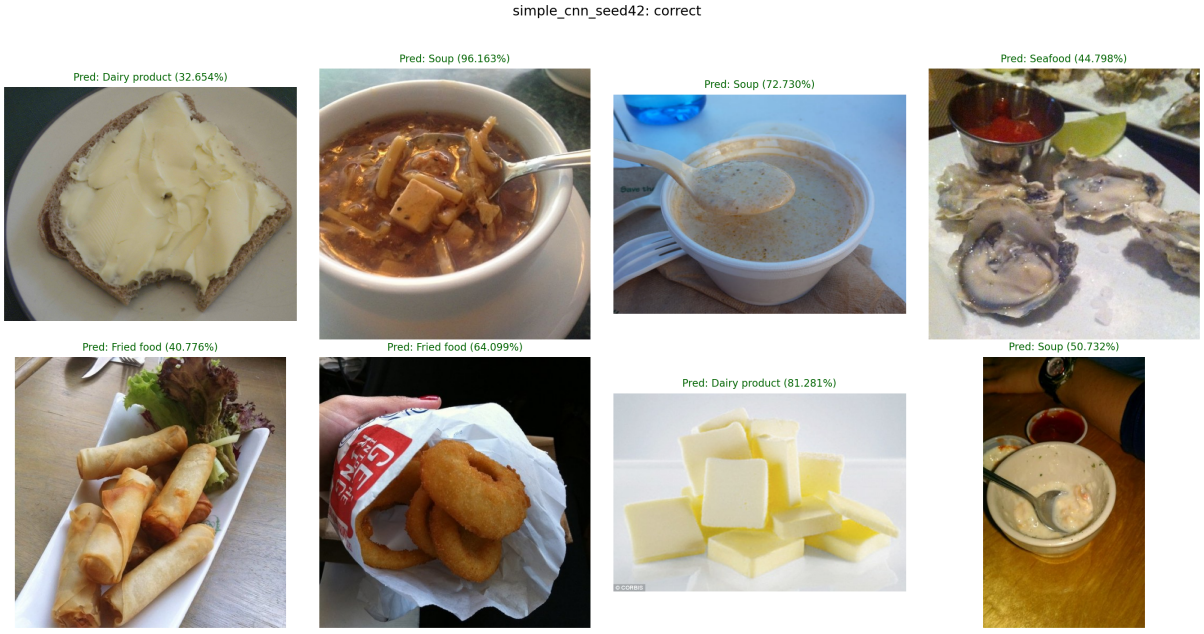

Figure 12: Correct predictions by SimpleCNN for training seed 42.

Figure 13: Correct predictions by ResNet-18 with a frozen backbone for training seed 42.

Figure 14: Correct predictions by fully fine-tuned ResNet-18 for training seed 42.

SimpleCNN correctly labels a slice of buttered bread as Dairy product with a softmax score of 32.654% and a bowl of soup as Soup with a score of 96.163%. Frozen ResNet-18 correctly identifies mussels as Seafood (99.568%), while fully fine-tuned ResNet-18 correctly identifies a plate of pasta as Noodles-Pasta (99.987%). These examples illustrate the range of scores that can accompany a correct prediction.

3.3.2. Difficult but Correct Predictions

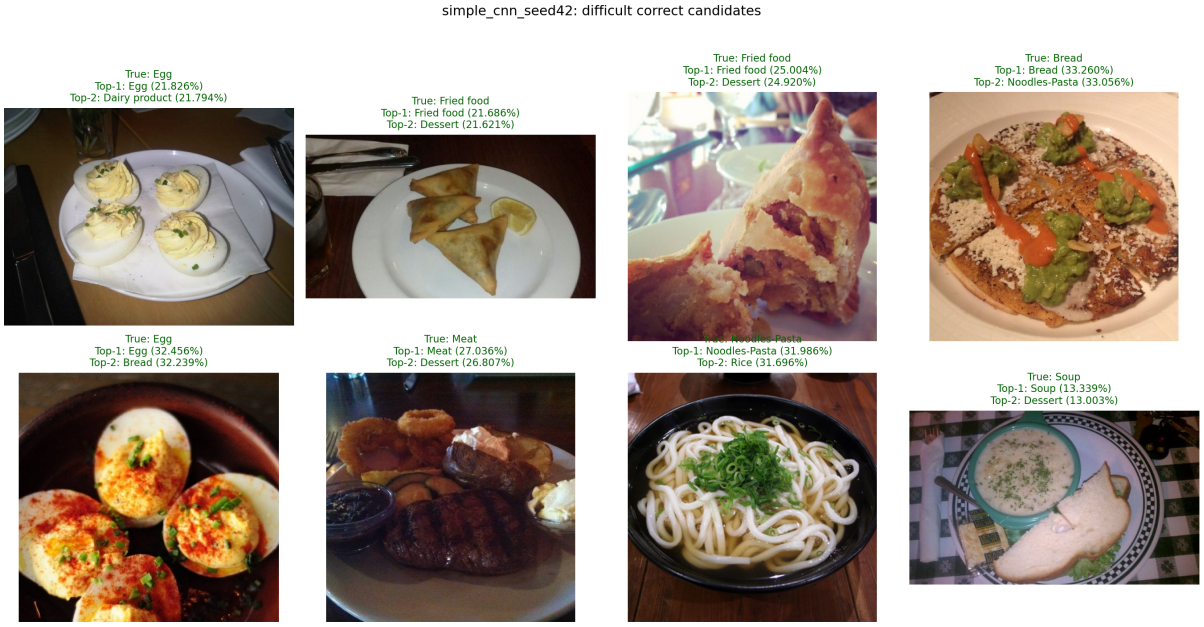

Figure 15: Correct predictions with small top-two softmax score gaps for SimpleCNN, training seed 42.

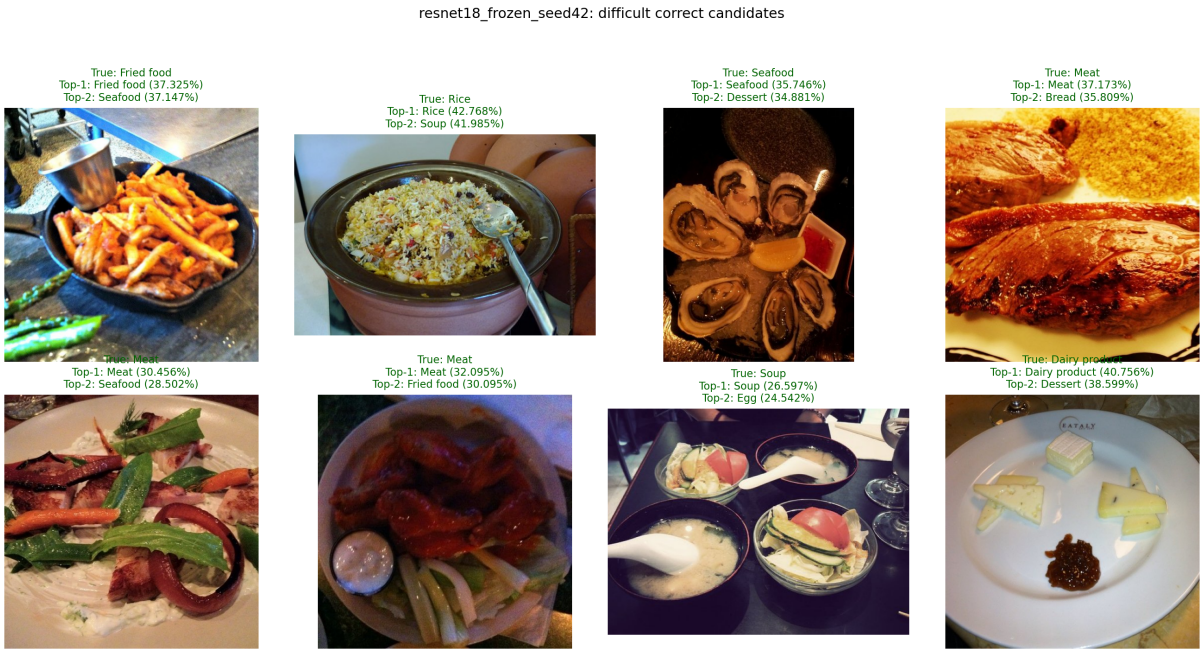

Figure 16: Correct predictions with small top-two softmax score gaps for ResNet-18 with a frozen backbone, training seed 42.

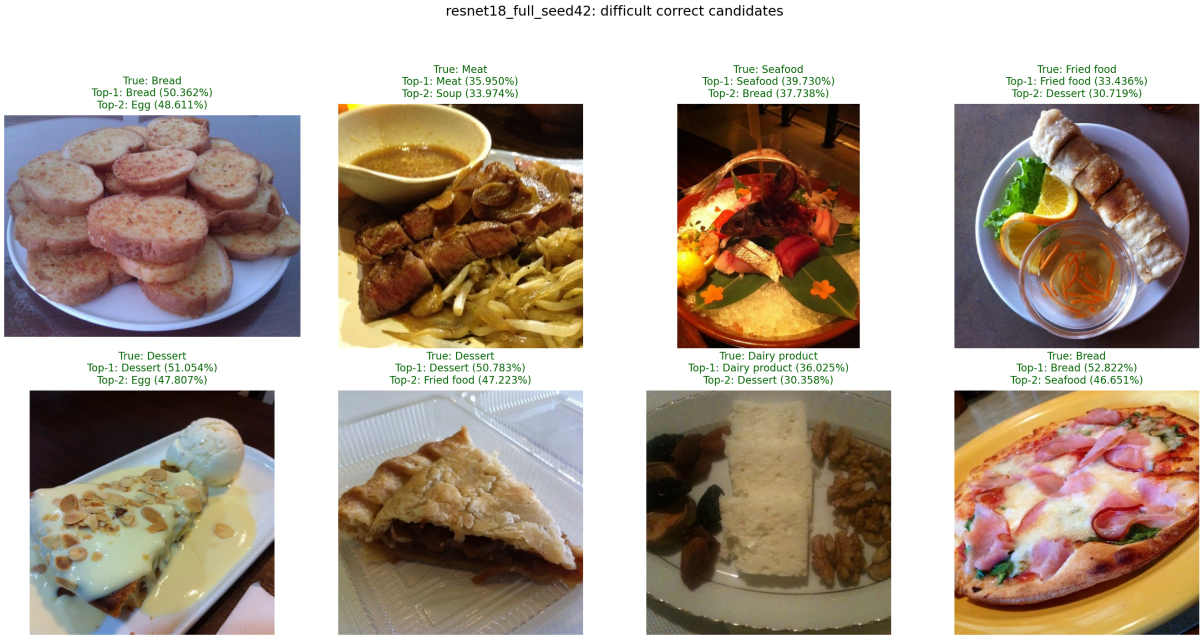

Figure 17: Correct predictions with small top-two softmax score gaps for fully fine-tuned ResNet-18, training seed 42.

These examples are selected from correct predictions with the smallest gaps between the two highest softmax scores. SimpleCNN ranks Egg (21.826%) just ahead of Dairy product (21.794%) for an image of deviled eggs. Frozen ResNet-18 ranks Fried food (37.325%) just ahead of Seafood (37.147%) for a plate of fries. Fully fine-tuned ResNet-18 ranks Bread (50.362%) ahead of Egg (48.611%) for sliced bread. The gaps indicate how close each model came to choosing its second-ranked class.

3.3.3. *Failure Cases*

Figure 18: Misclassified test images from SimpleCNN for training seed 42.

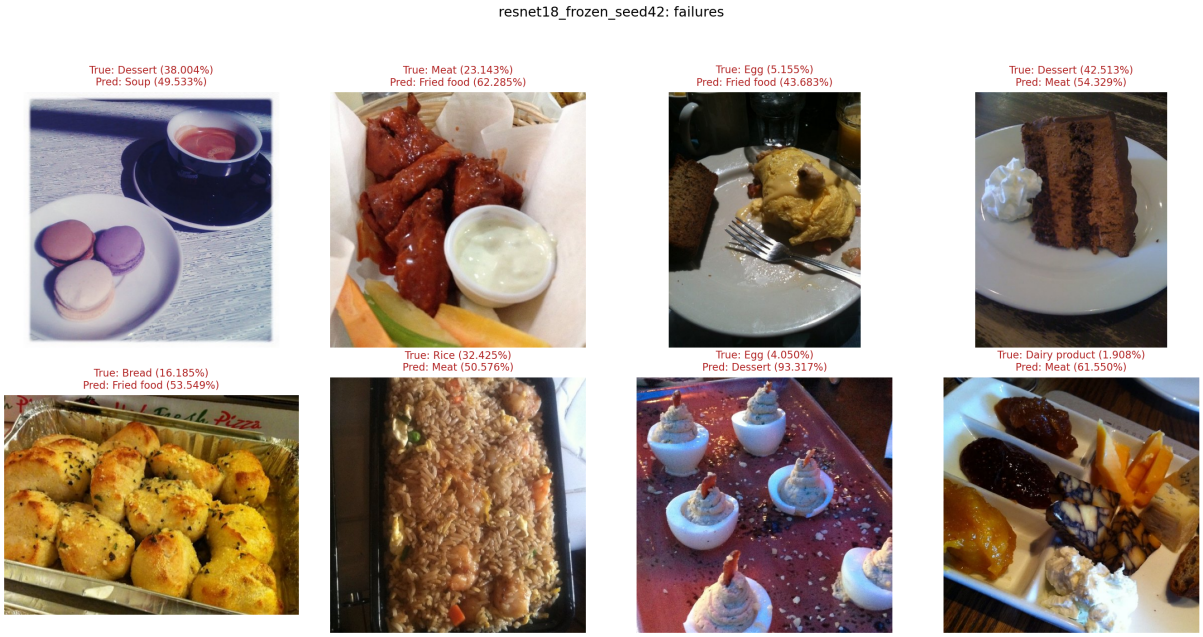

Figure 19: Misclassified test images from ResNet-18 with a frozen backbone for training seed 42.

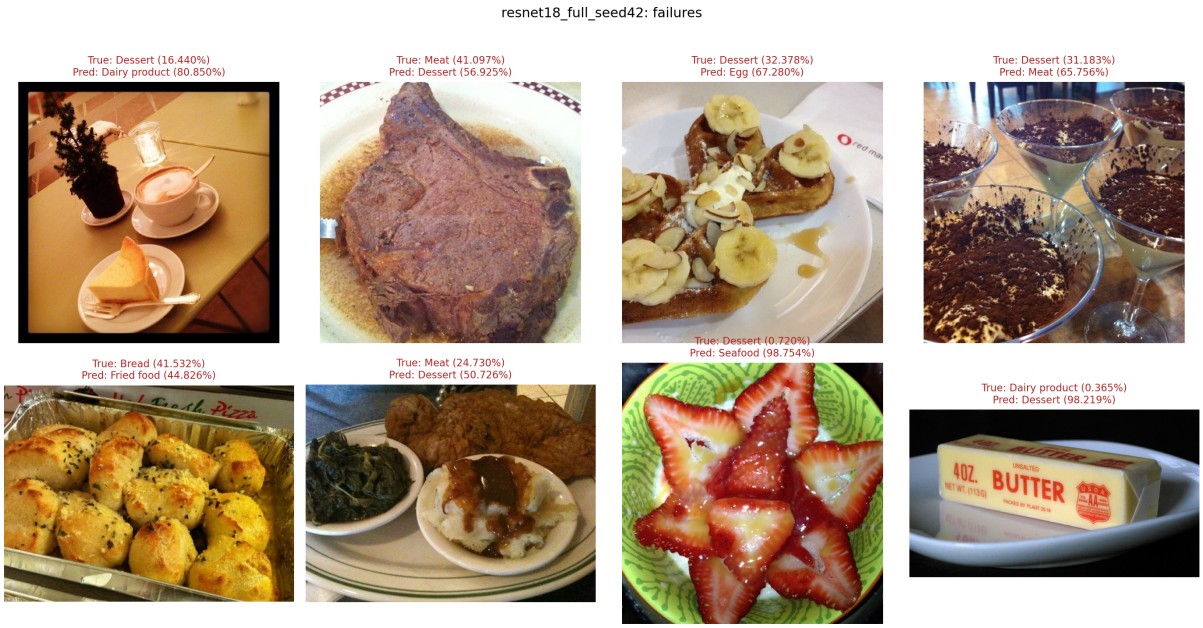

Figure 20: Misclassified test images from fully fine-tuned ResNet-18 for training seed 42.

SimpleCNN labels a fried-rice dish as Noodles-Pasta (45.938%), illustrating the Rice-to-Noodles-Pasta confusion in Figure 9. Both ResNet-18 variants label the same tray of bread rolls as Fried food, with scores of 53.549% for the frozen model and 44.826% for the fully fine-tuned model. The fully fine-tuned model also labels packaged butter as Dessert with a score of 98.219%.

3.4. Comparison and Discussion

3.4.1. Overall Predictive Performance

The gains in Table 14 follow the same ordering under top-1 accuracy and Macro-F1. The ranking therefore holds both when test images are weighted equally and when each class contributes equally. Full fine-tuning improves mean test Macro-F1 over the frozen configuration by 6.41 %.

3.4.2. Class-Specific Improvements and Remaining Errors

Rice gains the most from SimpleCNN to frozen ResNet-18, with mean F1 rising from 43.00% to 92.39%. Egg shows the largest further gain after full fine-tuning, from 79.13% to 91.20%. Dairy product remains the lowest-recall class. The packaged-butter example in Figure 20 illustrates its recurring confusion with Dessert.

3.4.3. Architecture and Input Representation

All configurations receive the same $3 \times 256 \times 256$ RGB representation after aspect-ratio-preserving resize, black padding, and ImageNet normalization. SimpleCNN learns its features from scratch through four convolutional blocks followed by global average pooling and a linear classifier. For each matched

seed, both ResNet-18 variants start from the same ImageNet-pretrained backbone weights and the same randomly initialized classification head. The frozen results show that these pretrained features already support Food-11 classification. The further gain from full fine-tuning supports adapting the backbone under the evaluated training policy.

3.4.4. Loss, Training Behavior, and Prediction Confidence

SimpleCNN continues to improve near the end of the 25-epoch budget, while frozen ResNet-18 largely plateaus. Full fine-tuning reaches nearly perfect training accuracy and shows a persistent training–validation gap, yet achieves the lowest mean test cross-entropy among the evaluated configurations.

The qualitative results illustrate the distinction between prediction correctness and model confidence. The fully fine-tuned model assigns a softmax score of 98.219% to Dessert for an image of packaged butter whose true class is Dairy product. A calibration analysis would assess how closely such scores correspond to the observed frequency of correct predictions.

3.4.5. Accuracy, Execution Time, and Parameter Trade-offs

SimpleCNN is the smallest and fastest configuration at inference. The frozen ResNet-18 trains only 5,643 parameters but still runs the entire backbone at inference, giving it an inference time close to the fully fine-tuned model. Full fine-tuning takes approximately 60.3 minutes per run including validation, compared with approximately 49.5 minutes for the frozen model. The additional training time accompanies a 6.41-percentage-point improvement in mean test Macro-F1.

3.4.6. Hypothesis Assessment and Limitations

The comparison supports the prediction that the two pretrained ResNet-18 configurations outperform SimpleCNN on Food-11. The gain reflects a comparison of complete configurations that differ in architecture, capacity, initialization, and training policy. Within ResNet-18, the comparison holds the architecture, pretrained weights, and initial head fixed for each seed. It supports full fine-tuning under the evaluated learning rates and backbone update policy. The results describe three training seeds on the same cleaned dataset partitions.

3.5. Error Analysis

3.5.1. Confusion Between Visually Similar Classes

Fully fine-tuned ResNet-18 misclassified 242 of the 2,770 Food-11 test images (8.74%) for seed 42. Each error was assigned to the first applicable group, using the true and predicted labels and applying G1–G5 in order. Table 18 summarizes these groups. The proposed causes are hypotheses informed by visual inspection of selected failure images.

Table 18: Error groups among the 242 incorrect test predictions of fully fine-tuned ResNet-18, seed 42. Percentages are relative to all errors.

Error group	**Errors**	**Main pairs** (True → predicted)	**Hypothesized cause**	**Improvement idea**
G1. Dairy–Dessert overlap	28 (11.6%)	Dairy product → Dessert (17) Dessert → Dairy product (11)	Shared ingredients, creamy textures, and similar serving styles may blur the boundary between dairy products and desserts. Accompanying foods can further obscure the intended class.	Add training examples of dairy products and visually similar desserts in varied contexts. Review labeling consistency for mixed dairy dishes.
G2. Bread in mixed dishes	71 (29.3%)	Bread → Fried food (13) Egg → Bread (12) Bread → Egg (10) Bread → Meat (9) Dessert → Bread (7)	Fillings, toppings, and accompanying foods provide competing class cues. A single image-level label does not explicitly identify the intended main component.	Review the rule for selecting the main food. Compare full-image and main-dish crops to assess the influence of accompanying ingredients.
G3. Other classes predicted as Dessert	37 (15.3%)	Fried food → Dessert (11) Meat → Dessert (9) Seafood → Dessert (6)	Preparation-based categories can overlap with Dessert. Similar colors, glossy surfaces, and plating styles may also make other foods resemble desserts.	Review labels for sweet fried foods. Add visually similar foods from competing classes and test dish-focused crops.
G4. Desserts predicted as other classes	35 (14.5%)	Dessert → Egg (9) Dessert → Meat (9) Dessert → Soup (6) Dessert → Seafood (5)	Desserts vary substantially in appearance. Color, texture, sauces, and ingredients may resemble other categories. Ambiguous annotations may also contribute to some errors.	Audit ambiguous labels and expand training coverage of visually diverse desserts, especially those resembling other categories.
G5. Ingredient and preparation overlap	35 (14.5%)	Fried food → Meat (7) Meat → Fried food (4) Meat → Egg (4)	Ingredient-based and preparation-based labels can describe the same dish. Coatings may hide ingredients, while mixed dishes contain cues associated with multiple classes.	Review category precedence for foods satisfying multiple labels. Add consistently labeled examples of coated foods and mixed dishes.
Other confusions	36 (14.9%)	Remaining pairs	These errors span several class pairs. The available examples do not establish a common explanation.	Inspect recurring pairs individually before selecting a targeted intervention.
Total	**242**			

The following figures show four selected failures from each error group in Table 18. These examples illustrate possible sources of confusion, but do not establish the causes of every error in a group.

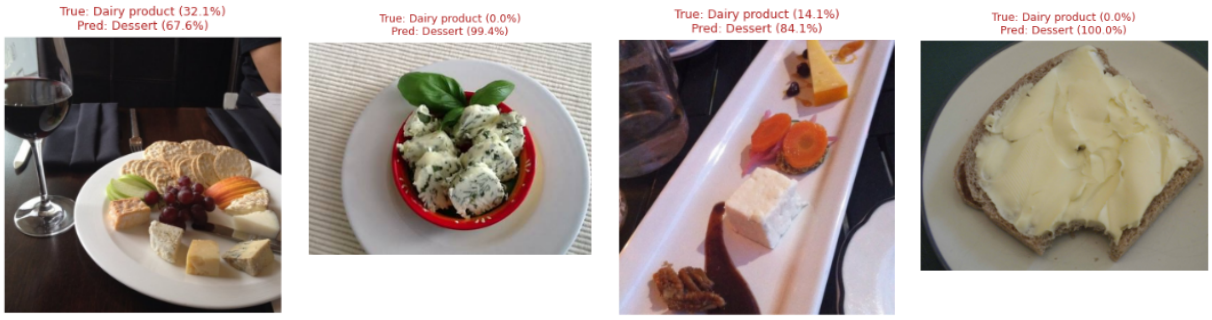

Figure 21: G1: Dairy product images predicted as Dessert, including mixed cheese plates and spread-covered bread.

Figure 22: G2: Mixed dishes containing bread alongside eggs or meat, illustrating competing class cues within a single image.

Figure 23: G3: Fried food, Seafood, and Meat images predicted as Dessert. The churro examples illustrate overlap between preparation-based and dessert categories.

Figure 24: G4: Dessert-labeled images predicted as Meat, Egg, or Soup. The second example contains a visible egg and warrants review of its annotation.

Figure 25: G5: Confusion involving Fried food, Meat, and Egg, with overlapping preparation and ingredient cues.

3.5.2. Incorrect Predictions with High Softmax Scores

Description and frequency: Some incorrect predictions strongly favor the wrong class rather than show ing a small difference between competing labels. This group overlaps with the class-based groups above

Examples: In Figure 21, the second image is labeled Dairy product but predicted as Dessert with a softmax score of 99.4%, while its true class receives 0.0% at the displayed precision. In Figure 25, the third image is labeled Meat but predicted as Fried food with 100.0%, while Meat receives 0.0% at the displayed precision. These cases illustrate that a dominant softmax score does not guarantee agreement with the ground-truth label.

Hypothesized cause: A model may learn visual features that strongly support an incorrect category for a particular image. Cross-entropy training does not guarantee that a high softmax score corresponds to a correct prediction. However, these failures alone do not establish systematic overconfidence or identify the specific features responsible.

Improvement ideas: Temperature scaling fitted on the validation set could be evaluated on the test set using calibration metrics. This method adjusts softmax scores without changing the predicted class.

3.6. Ablation Study

The ablation compares training with horizontal flipping ($p = 0.5$) and training with the flip transform removed ($p = 0$). For each model, runs are paired by training seed and use the same cleaned partitions. Test Macro-F1 is the primary metric.

Model	Top-1 Accuracy (%)	Macro-Precision (%)	Macro-Recall (%)	Macro-F1 (%)	Test CE loss
SimpleCNN	60.01 ± 1.47	62.41 ± 0.91	56.79 ± 1.43	57.09 ± 1.81	1.2211 ± 0.0196
ResNet-18 (frozen)	84.33 ± 0.06	85.50 ± 0.04	84.52 ± 0.11	84.85 ± 0.11	0.4988 ± 0.0022
ResNet-18 (full)	$\mathbf{90.76} \pm 0.07$	$\mathbf{91.12} \pm 0.14$	$\mathbf{91.18} \pm 0.04$	$\mathbf{91.12} \pm 0.05$	$\mathbf{0.3423} \pm 0.0087$

Table 19: Test performance without horizontal flipping, reported as mean ± sample standard deviation across training seeds 42, 43, and 44. Bold values indicate the best mean for each metric.

For each model and seed, the paired difference is the augmented result minus the corresponding result without augmentation. Positive differences favor augmentation for accuracy and the macro-averaged metrics. Negative differences favor augmentation for cross-entropy loss.

Model	Δ Top-1 Accuracy (%)	Δ Macro-Precision (%)	Δ Macro-Recall (%)	Δ Macro-F1 (%)	Δ Test CE loss
SimpleCNN	$+0.39 \pm 0.50$	$+1.07 \pm 2.38$	$+0.44 \pm 1.12$	$+0.49 \pm 1.30$	-0.0151 ± 0.0156
ResNet-18 (frozen)	$+0.29 \pm 0.19$	-0.05 ± 0.06	$+0.71 \pm 0.22$	$+0.41 \pm 0.16$	-0.0051 ± 0.0019
ResNet-18 (full)	$+0.61 \pm 0.38$	$+0.53 \pm 0.41$	$+0.56 \pm 0.42$	$+0.55 \pm 0.42$	-0.0227 ± 0.0106

Table 20: Paired test differences computed as Augmented minus No Augmentation within each training seed. Results are reported as mean ± sample standard deviation across seeds 42, 43, and 44. Differences in accuracy and macro-averaged metrics are in percentage points (%).

Horizontal flipping improves test Macro-F1 for both ResNet-18 configurations across all three seeds. The mean gains are 0.41 % for the frozen backbone and 0.55 for full fine-tuning. For the frozen model, the Macro-F1 gain accompanies a 0.71-percentage-point increase in Macro-Recall and a 0.05-percentage-point decrease in Macro-Precision. For SimpleCNN, the Macro-F1 difference changes sign across seeds, and its sample standard deviation of 1.30 % exceeds the mean gain of 0.49. The effect on SimpleCNN is inconclusive.

3.7. Limitations and Conclusion

3.7.1. Data Limitations

The original training split is imbalanced. The largest classes contain about 5.36 times as many images as Rice. Macro-F1 gives each class equal weight, but per-class scores remain more sensitive to individual errors when fewer test images are available. For example, the test set contains 90 Rice images and 440

Dessert images.

Food-11 uses broad, visually overlapping categories and assigns one label per image. Mixed dishes and ambiguous annotations can make individual errors difficult to interpret. Some classes also differ in photographic style, such as studio backgrounds in Dairy product and Vegetable/Fruit images. Models may use these cues alongside the appearance of the food. The frequency of ambiguous or off-topic annotations was not quantified.

All runs use the same cleaned partitions, so the three seeds measure training variability on one fixed split. Duplicate removal reduces overlap within and across these partitions. The available metadata provides food-class labels and lacks subject, restaurant, plate, and scene identifiers for grouped splitting.

3.7.2. Method limitations.

The comparison between SimpleCNN and pretrained ResNet-18 combines differences in architecture, parameter count, and initialization. A matched architecture comparison would be needed to estimate the contribution of pretraining alone. Within ResNet-18, the results compare two specific training policies under the chosen learning rates and 25-epoch budget. SimpleCNN's training loss is still decreasing near the end of this budget, leaving open how much it could improve with longer training.

The horizontal-flip ablation uses three matched seeds. More seeds would help estimate how consistently the small observed gains recur.

3.7.3. Generalization and Future Work

Evaluation on an independent food-image dataset would help determine whether the model ranking holds across image sources, cuisines, and presentation styles. Comparing randomly initialized and ImageNet-pretrained ResNet-18 under a matched training setup would help isolate the contribution of pretraining. Longer runs with validation-based checkpoint selection could also clarify how much the training budget limits SimpleCNN. A review of Dairy product images predicted as Dessert could help separate ambiguous scenes from recurring model errors.

3.7.4. Conclusion

The first hypothesis, that pretrained ResNet-18 would outperform the from-scratch SimpleCNN, is supported by Table 14. Mean test Macro-F1 rises from 57.58% for SimpleCNN to 85.26% for frozen ResNet-18 and 91.67% for fully fine-tuned ResNet-18. The 6.41-percentage-point gain over the frozen configuration supports updating the backbone under the evaluated fine-tuning policy.

The second hypothesis, that horizontal flipping would improve test Macro-F1, is supported for both ResNet-18 configurations and remains inconclusive for SimpleCNN. In Table 20, both ResNet-18 con-

figurations improve across all three seeds, with paired gains of $+0.41 \pm 0.16$ percentage points for the frozen model and $+0.55 \pm 0.42$ for full fine-tuning. SimpleCNN has a mean gain of $+0.49$ %, with differences of both signs and a sample standard deviation of 1.30 %. Fully fine-tuned ResNet-18 with horizontal flipping achieves the highest mean test Macro-F1 among the evaluated configurations.

4. AI Usage Disclosure

The group takes full responsibility for all submitted code, numerical results, figures, claims, and text. AI tools supported learning, reviewing and refining member-written code and drafts, building skeleton pages, and language editing. Group members reviewed the adopted suggestions and verified them against the implementation, experimental outputs, and assignment requirements. All reported experimental results were obtained from runs executed by the group, and result figures were produced from those outputs. AI assistance with code or presentation did not replace the execution of experiments or provide fabricated data or results. The following records describe the scope of AI assistance and the corresponding student verification.

4.1. Review and Refinement of the Training and Evaluation Pipeline

- **Tool:** GPT-6.1 Sol.
- **Used by:** Lê Nguyễn Gia Phúc.
- **Development stage:** Pipeline review, refinement, and debugging (Milestone 1).
- **Task:** Review the member-written Food-11 classification pipeline for SimpleCNN, frozen ResNet-18, and fully fine-tuned ResNet-18. Document how to reconstruct the cleaned dataset splits, evaluate saved checkpoints, and reproduce the reported results.
- **Prompt summary:**

  Provided an existing member-written pipeline and specified the required workflow, current implementation of EDA, train/validation/test splitting, preprocessing and augmentation, model training pipeline, evaluation metrics, checkpoint selection, and reproducibility. Requested a review of the workflow, explanations of technical issues, and suggestions for improving reproducibility. Also requested appropriate ways to organize and save checkpoints, configurations, experimental outputs.

- **AI contribution:** Reviewed the supplied pipeline and suggested improvements to data preparation, training, evaluation, checkpoint selection, and reproducibility. Recommended saving model configurations, dataset split indices, seed settings, dependency versions, hardware information, checkpoints, metric summaries, and training histories. Helped modify visualization code for learning curves, confusion matrices, and prediction examples.
- **Student verification:** Compared the suggested changes with the original member-written pipeline and the experimental requirements. Reviewed the data split, preprocessing, training settings, metric calculations, checkpoint-selection rule, and reproducibility settings. Executed the training workflow and inspected the generated metrics and checkpoints.

- **Affected files/sections:** Assignment2/Assignment2.ipynb.
- **Responsible member:** Lê Nguyễn Gia Phúc.

4.2. Model Architecture Review and Technical Verification

- **Tool:** GPT-6.1 Sol
- **Development stage**: Model Implementation (Milestone 1)
- **Used by:** Lê Nguyễn Gia Phúc
- **Task:** Review the Food-11 implementations of SimpleCNN, frozen ResNet-18, and fully fine-tuned ResNet-18. Check tensor dimensions, residual connections, classifier heads, and trainable parameter counts.
- **Prompt summary:** The member provided the model implementations and draft architecture descriptions. The prompts asked how the models process $3 \times 256 \times 256$ RGB inputs, where downsampling occurs in ResNet-18, and how the shortcut and residual branches match in shape before addition.
- **AI contribution:** Checked the ResNet-18 description against the TorchVision source and explained the stem, the two convolutions in each residual branch, and the stride-2 projection shortcuts at the start of Stages 2, 3, and 4. Helping to revise the architecture description
- **Student verification:** The member compared their original ResNet-18 description with provided explanation and checked both against the official TorchVision documentation and source code
- **Affected files/sections:** Assignment2/Assignment2.ipynb, Report(Methodology)
- **Responsible member:** Lê Nguyễn Gia Phúc.

4.3. Feedback on Results Interpretation and Discussion

- **Tool:** GPT-6.1 Sol
- **Development stage**: Report writing (Milestone 3)
- **Used by:** Nguyễn Việt Hùng, Hoàng Xuân Bách, Lê Nguyễn Gia Phúc.
- **Task:** Reviewing the student's initial interpretations and suggesting improvements to the organization and focus of the comparative discussion.
- **Prompt summary:** Provided the assignment's discussion requirements, the experimental results already obtained, and an initial student-written draft explaining those results. Requested feedback on the draft and suggestions on how to organize and refine the explanations in the report, linking

relevant comparisons, observed patterns, and supporting evidence to each requirement. The aim was to keep the discussion focused on the specification and available evidence, while avoiding repetitive descriptions, irrelevant commentary, and unsupported conclusions.

- **AI contribution:** Reviewed the supplied draft and suggested revisions to its structure, wording, and interpretation of the results. Helped connect overall performance, class-specific differences, confusion patterns, training behavior, and computational trade-offs to the assignment requirements. Suggested qualifying explanations related to architecture, regularization, and training where the experiments did not isolate their individual effects.
- **Student verification:** Compared the suggested revisions with the original draft and the assignment's discussion requirements. Checked numerical comparisons against notebook outputs and result tables, and assessed interpretations against the learning curves, confusion matrices, and prediction examples.
- **Affected files/sections:** Report(Implementation Results)
- **Responsible member:** Nguyễn Việt Hùng, Hoàng Xuân Bách, Lê Nguyễn Gia Phúc.

4.4. English Language Editing

- **Tool:** GPT-6.1 Sol.
- **Development stage**: Final Revision (Milestone 3)
- **Used by:** Hoàng Xuân Bách,Lê Nguyễn Gia Phúc, Nguyễn Việt Hùng
- **Task:** Improving the grammar, clarity, and readability of student-written explanations in the report.
- **Prompt summary:** Provided draft paragraphs written by the student and requested sentence-level language editing: "Fix the grammar and improve the clarity of this paragraph without changing its meaning, technical terminology, or numerical results."
- **AI contribution:** Suggested corrections to grammar, sentence structure, word choice, and transitions. Proposed clearer wording for descriptions of the SimpleCNN, frozen Resnet18 and Resnet18 full fine-tuning results. The requested scope was language editing of the supplied text, with the original interpretations and numerical values preserved.
- **Student verification:** Compared the suggested edits with the original student-written paragraphs. Checked that model names, technical terminology, numerical values, units, and comparisons remained consistent with the experimental results.
- **Affected files/sections:** Report, Readme.md, assignment2.html

- **Responsible member:** Hoàng Xuân Bách,Lê Nguyễn Gia Phúc, Nguyễn Việt Hùng

4.5. AI-Assisted Github Pages Skeleton

- **Tool:** GPT-5.6 Sol.
- **Development stage**: Planning (Milestone 1)
- **Used by:** Hoàng Xuân Bách, Lê Nguyễn Gia Phúc, Nguyễn Việt Hùng.
- **Task:** Create the initial HTML/CSS skeleton for the group website, including a landing page, assignment pages, and reusable table styles.
- **Prompt summary:** Requested a landing page layout and a basic HTML/CSS skeleton with placeholder sections, sample table markup, and shared styles for table headers, cells, and captions.
- **AI contribution:** The tool suggested the initial page structure and navigation layout. It also provided sample table markup and reusable CSS for consistent table formatting across the assignment pages.
- **Student verification:** Reviewed and revised the generated skeleton against the project requirements. Members added the course details, team profiles, and assignment descriptions. Also checked the semantic HTML structure, navigation links, table headers and captions, and how the layouts displayed at different screen sizes.
- **Affected files/sections:** index.html, style.css, assignment1.html, assignment2.html, assignment3.html.
- **Responsible members:** Hoàng Xuân Bách, Lê Nguyễn Gia Phúc, Nguyễn Việt Hùng.

4.6. Consistency Verification Between Experimental Outputs and Report

- **Tool:** GPT-6.1 Sol.
- **Development stage**: Final Revision (Milestone 3)
- **Used by:** Lê Nguyễn Gia Phúc.
- **Task:** Supporting the final consistency check between experimental outputs and the report's implementation results.
- **Prompt summary:** Provided the final experimental outputs and the corresponding report content. Requested assistance with checking numerical consistency and identifying outdated values, mismatched labels, or explanations that were not supported by the results.

- **AI contribution:** Assisted with reviewing metric values, mean and sample standard deviation, parameter counts, computational costs, and Baseline/NoAugmentation comparisons. Suggested checking that tables, figures, captions, and conclusions referred to the same experimental configurations and aggregation conventions.
- **Student verification:** Compared the report's numerical statements, tables, figures, and captions with the final notebook outputs. Checked seed settings, variant labels, units, and aggregation conventions. Corrected outdated or inconsistent content and reviewed the conclusions against the experimental evidence.
- **Affected files/sections:** Report(Implementation Results)
- **Responsible member:** Lê Nguyễn Gia Phúc.

4.7. Consistency Verification Between Experimental Outputs and GitHub Pages

- **Tool:** GPT-6.1 Sol.
- **Development stage**: Final Revision (Milestone 3)
- **Used by:** Hoàng Xuân Bách.
- **Task:** Supporting the final consistency check between experimental outputs and the results presented on GitHub Pages.
- **Prompt summary:** Provided the final experimental outputs and the corresponding GitHub Pages content. Requested assistance with checking whether the published numerical summaries, tables, figures, and explanations accurately represented the completed experiments.
- **AI contribution:** Assisted with reviewing the consistency of displayed metrics, model names, variant labels, units, and figure descriptions. Suggested checking that published figures and summaries used the final results, with single-seed examples distinguished from results aggregated across three seeds.
- **Student verification:** Compared the Assignment 2 results displayed on GitHub Pages with the final notebook outputs. Checked numerical summaries, table entries, figure versions, captions, and model labels. Corrected outdated content and inspected the rendered pages to confirm that the updated results and figures were displayed correctly.
- **Affected files/sections:** assignment2.html
- **Responsible member:** Hoàng Xuân Bách.

4.8. LaTeX Formatting Assistance

- **Tool:** GPT-6.1 Sol.
- **Used by:** Lê Nguyễn Gia Phúc, Hoàng Xuân Bách, Nguyễn Việt Hùng
- **Development stage:** Report formatting and final revision (Milestone 3).
- **Task:** Improving the LaTeX presentation of student-provided report content.
- **Prompt summary:** Provided existing text, tables, and equations and requested formatting assistance. Requests included converting the experimental design into a comparison table, adjusting column widths and text alignment, splitting long tables, and formatting mathematical expressions without changing their intended meaning or numerical values.
- **AI contribution:** Suggested LaTeX environments and formatting code for tables, equations, and structured lists. Helped arrange the main comparison and ablation study side by side, merge shared experimental settings, and improve the readability of lengthy tables. Provided revised LaTeX snippets based on the supplied content.
- **Student verification:** Compiled the revised LaTeX and inspected the rendered report. Compared the formatted content with the original text and experimental outputs. Checked table entries, mathematical symbols, captions, labels, and cross-references, and adjusted spacing and page breaks where necessary.
- **Affected files/sections:** Report(LaTeX source code)
- **Responsible member:** Lê Nguyễn Gia Phúc, Hoàng Xuân Bách, Nguyễn Việt Hùng